

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Architettura Modulare per l'Arricchimento Dati in Sistemi di Raccomandazione Libraria

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

CANDIDATO

Cristian Porzio

Matricola: 0512113801

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel



Serve fare tutte le scelte per farne una soltanto

Abstract

I sistemi di raccomandazione libraria svolgono un ruolo cruciale nell'orientare i lettori all'interno di un'offerta editoriale in continua espansione. Tuttavia, la loro efficacia è spesso compromessa da dataset incompleti o statici, che riducono la pertinenza dei suggerimenti.

Questa tesi affronta il problema attraverso la progettazione e l'implementazione di un'architettura modulare per l'arricchimento dei metadati, basata su un insieme di provider estensibili. Tale architettura consente l'integrazione di fonti eterogenee e garantisce una maggiore completezza e consistenza del corpus bibliografico.

Il motore di raccomandazione sfrutta questi dati arricchiti mediante un approccio ibrido che combina tecniche content-based e collaborative filtering, potenziate dall'impiego di strutture per la ricerca approssimata come Annoy e FAISS. La personalizzazione è ulteriormente raffinata attraverso la profilazione dell'utente e un re-ranking guidato da fattori come il numero di pagine e i generi preferiti.

Il risultato è un ecosistema software modulare, estensibile e manutenibile, dotato sia di CLI che di interfaccia web minimale in grado dimostrare l'efficacia di strategie basate su architetture componibili e tecniche NLP moderne per affrontare le sfide di qualità dei dati nell'ambito dei sistemi di raccomandazione.

Indice

Elenco delle Figure **iv**

Elenco delle Tabelle **v**

1 Introduzione **1**

1.1 Contesto Applicativo 1

1.2 Motivazioni e obiettivi 2

1.3 Risultati ottenuti 4

1.4 Struttura Tesi 4

2 Background e Stato dell'Arte **5**

2.1 Cos'è un libro? 5

2.1.1 Generi Letterari 6

2.2 Algoritmi di Raccomandazione 8

2.2.1 Importanza degli Algoritmi in numeri 8

2.2.2 Cenni sulle Metodologie 10

2.3 Piattaforme di Raccomandazione Esistenti 13

2.3.1 Goodreads 13

2.3.2 TheStoryGraph 14

2.4 Data Augmentation 15

2.4.1 Obiettivi e Benefici 15

2.4.2 Approcci 16
2.4.3 Difficoltà 16

3 Applicazione Sviluppata **17**
3.1 Contesto e Obiettivi del Progetto 17
3.2 Metodologia e Fasi di di Sviluppo 17
3.2.1 Fasi Individuate 18
3.2.2 Tecnologie Impiegate 18
3.3 Acquisizione del Dataset 20
3.3.1 Pre-elaborazione e Ottimizzazione dei Dati (ETL) 21
3.3.2 Pulizia e Preparazione Dati 24
3.3.3 Analisi del Dataset 25
3.4 Sistema di Inizializzazione (Core) 30
3.4.1 Scelte Architetturali 30
3.4.2 Flusso di Inizializzazione 31
3.4.3 Il Ruolo del Core nel Processo di Data Ingestion 33
3.5 Sistema di Data Augmentation 35
3.5.1 Architettura a Provider 35
3.5.2 Aggregazione dei Metadati 36
3.5.3 Logica di Business e Database 37
3.5.4 Mapping dei Generi 38
3.6 Sistema di Raccomandazione 42
3.6.1 Architettura del Sistema di Raccomandazione 43
3.7 WebApp 47

4 Valutazione dell'Applicazione **49**
4.1 Sistema di Data Augmentation 49
4.1.1 Metodologia di Test e Dataset 49
4.1.2 Confronto Quantitativo con altri Providers 52
4.2 Sistema di Raccomandazione 54
4.2.1 Metodologia di Valutazione Offline 54
4.2.2 Valutazione Qualitativa tramite Beta-Testing 60

5 Conclusioni **64**

5.1 Sviluppi Futuri 65

Bibliografia **66**

Elenco delle figure

1.1 Lista dei 5 dataset più utilizzati su Kaggle in materia. 3
1.2 Distribuzione dei libri per anno di pubblicazione. 3

2.1 Copertina di un libro 6
2.2 Corpo dell'opera 6
2.3 Visuale gerarchia di tecniche di raccomandazione. 10

3.1 Distribuzione delle pubblicazioni negli anni. 25
3.2 Distribuzione dei generi. 26
3.3 Distribuzione stelle nelle recensioni 27
3.4 Pagine di Sign-Up e Ricerca 47
3.5 Catalogo personale 48
3.6 Visualizzazione delle Raccomandazioni 48

4.1 Distribuzione dei generi 51
4.2 Distribuzione dell'Indice di Jaccard per tutti i libri in comune. 52
4.3 Grafici delle risposte ai questionari. 61

Elenco delle tabelle

2.1 Sintesi del mercato globale e italiano dell'Intelligenza Artificiale. . . . 9

3.1 Confronto tra dimensioni dei file JSON e corrispondenti collezioni nel database MongoDB. 22
3.2 Distribuzione dei dati mancanti nel dataset 28

4.1 Confronto delle performance (Collaborative Filtering, k=10) con diverse strategie di Re-Ranking. 56
4.2 Evoluzione delle metriche di performance durante il processo di ottimizzazione. 56
4.3 Confronto tra suddivisione casuale e temporale con analisi dell'allineamento di genere. 58

CAPITOLO 1

Introduzione

1.1 Contesto Applicativo

Negli ultimi cinque anni, il mercato editoriale globale ha superato i 2 milioni di nuovi titoli pubblicati annualmente[1], con una previsione di tasso di crescita del volume di vendite degli e-book e audio-libri interessante per i prossimi 5 anni, dimostrando che non sia un settore così carente [2]. Tuttavia, malgrado questa apparente abbondanza di offerta, emerge una dissonanza significativa: la percezione diffusa di una scarsa presenza di lettori, accentuata ulteriormente dalla rarefazione dei `lettori forti`.[3]

La difficoltà dei lettori nell'identificare opere in linea con i propri interessi rappresenta una problematica concreta e ampiamente riconosciuta. Anzi, potrebbe costituire un deterrente per l'avvicinamento o la prosecuzione dell'esperienza di lettura.

[1]UNESCO `https://en.wikipedia.org/wiki/Books_published_per_country_per_year`

[2]STATISTA `https://www.statista.com/outlook/amo/media/books/ebooks/worldwide?currency=USD`

[3]Lettore forte, chi legge annualmente molti più libri rispetto alla media[1].

1.2 Motivazioni e obiettivi

La presente tesi non si propone di sviluppare un sistema di raccomandazione completamente innovativo o rivoluzionario dal punto di vista algoritmico, soprattutto considerando le limitazioni di potenza computazionale a disposizione. L'obiettivo principale, infatti, è piuttosto quello di affrontare una problematica spesso trascurata ma cruciale: **l'arricchimento e aggiornamento dei dataset di libri utilizzati nei sistemi di raccomandazione**.

L'efficacia di un sistema di raccomandazione libraria dipende in modo critico dalla qualità del dataset sottostante.

Attualmente, molte ricerche accademiche e modelli si basano su dataset statici o poco aggiornati, come ad esempio quelli accessibili da Kaggle (Figura 1.1), che pur essendo molto ricchi, risultano datati, presentando un ritardo significativo nell'inclusione di nuove pubblicazioni, (Figura 1.2), dati mancanti e una categorizzazione spesso troppo semplificata[2]. Queste limitazioni compromettono la capacità di proporre contenuti aggiornati, riducendo il valore per l'utente finale.

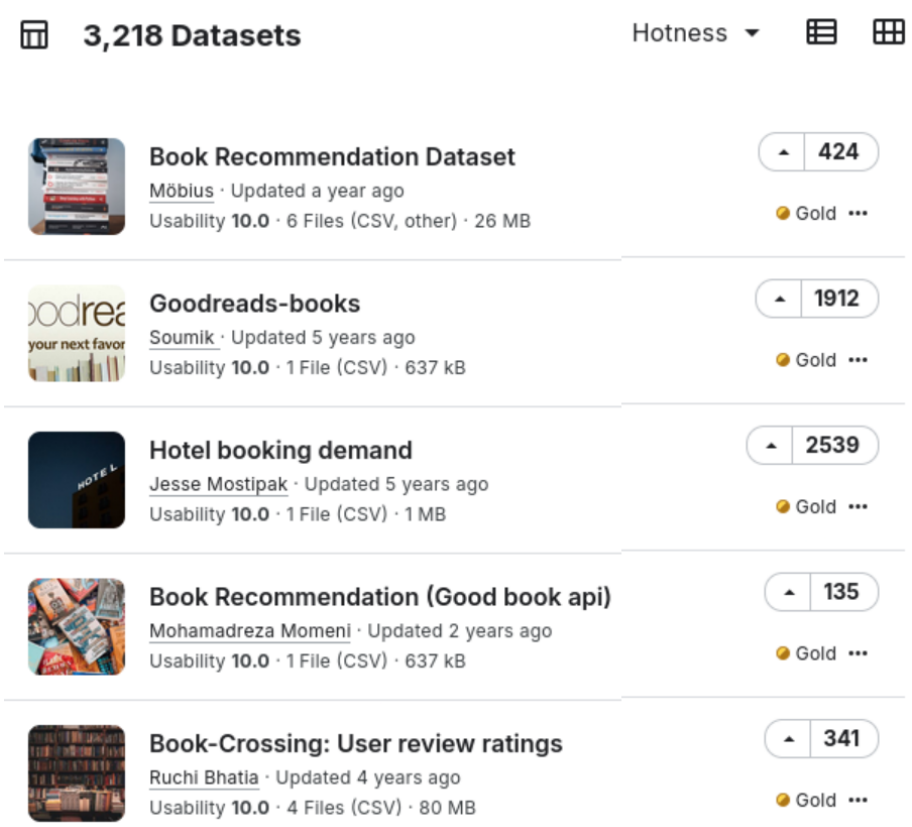


Figura 1.1: Lista dei 5 dataset più utilizzati su Kaggle in materia.

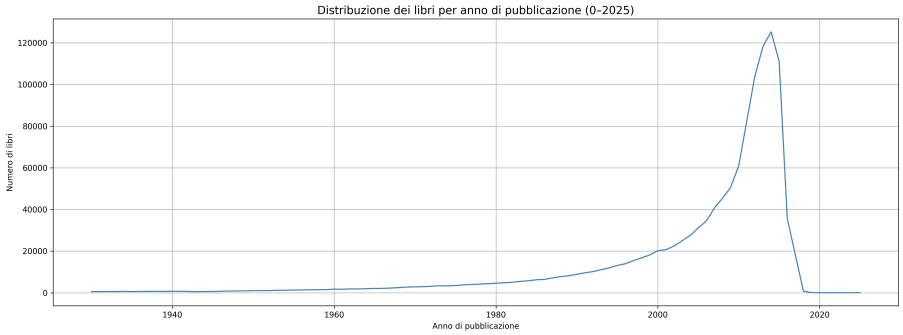


Figura 1.2: Distribuzione dei libri per anno di pubblicazione.

1.3 Risultati ottenuti

È stata progettata e testata una pipeline di arricchimento dati a due stadi — che aggrega informazioni da più provider e le normalizza tramite l'LLM Gemma-3 — dimostrando la fattibilità dell'approccio. Su questa base è stato realizzato un motore di raccomandazione autonomo e performante, che combina tecniche content-based e collaborative filtering e utilizza indici come Annoy e FAISS per garantire risposte rapide.

1.4 Struttura Tesi

La tesi è formata da 5 capitoli:

- **Introduzione**, in cui viene presentato il contesto, le motivazioni e gli obiettivi del lavoro.
- **Stato dell'Arte**, analizza le metodologie e tecniche di Data Augmentation e Raccomandazione.
- **Applicazione Sviluppata**, dove si descrive nel dettaglio il progetto, le metodologie e tecnologie.
- **Valutazione dell'Applicazione**, dove sono state eseguite le analisi delle performance dei sistemi.
- **Conclusioni**, con una sintesi dei risultati ottenuti e possibili sviluppi futuri.

CAPITOLO 2

Background e Stato dell'Arte

2.1 Cos'è un libro?

Nel contesto dei sistemi di raccomandazione e dell'informatica, un libro può essere descritto come un'entità composta da tre principali categorie di informazioni[3]:

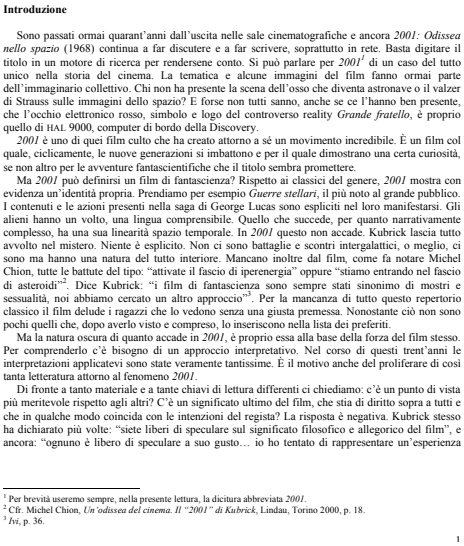
1. **Metadati**[1]: comprendono elementi come titolo, autore, editore, genere, lingua, data di pubblicazione, numero di pagine, e altri attributi bibliografici;
2. **Contenuto testuale**: il corpo principale dell'opera;
3. **Struttura**: rappresentata dal sommario, che evidenzia la gerarchia tra capitoli e sezioni, utile per analisi strutturali o tematiche.

Nei libri tradizionali, molti di questi elementi sono immediatamente riconoscibili dalla copertina (cfr. Figure 2.1, 2.2), come il titolo e l'autore. Tuttavia, a causa della varietà di formati, delle riedizioni e delle scelte editoriali, alcuni componenti, come gli indici od il genere, possono talvolta risultare assenti, impliciti o non standardizzati.

[1]Wikipedia `https://it.wikipedia.org/wiki/Metadato`



Figura 2.1: Copertina di un libro



Introduzione

Sono passati ormai quarant'anni dall'uscita nelle sale cinematografiche e ancora *2001: Odissea nello spazio* (1968) continua a far discutere e a far scrivere, soprattutto in rete. Basta digitare il titolo in un motore di ricerca per rendersene conto. Si può parlare per *2001*[1] di un caso del tutto unico nella storia del cinema. La tematica e alcune immagini del film fanno ormai parte dell'immaginario collettivo. Chi non ha presente la scena dell'osso che diventa astronave o il valzer di Strauss sulle immagini dello spazio? E forse non tutti sanno, anche se ce l'hanno ben presente, che l'occhio elettronico rosso, simbolo e logo del controverso reality *Grande fratello*, è proprio quello di HAL 9000, computer di bordo della Discovery.

2001 è uno di quei film culto che ha creato attorno a sé un movimento incredibile. È un film col quale, ciclicamente, le nuove generazioni si imbattono e per il quale dimostrano una certa curiosità, se non altro per le avventure fantascientifiche che il titolo sembra promettere.

Ma *2001* può definirsi un film di fantascienza? Rispetto ai classici del genere, *2001* mostra con evidenza un'identità propria. Prendiamo per esempio *Guerre stellari*, il più noto al grande pubblico. I contenuti e le azioni presenti nella saga di George Lucas sono espliciti nel loro manifestarsi. Gli alieni hanno un volto, una lingua comprensibile. Quello che succede, per quanto narrativamente complesso, ha una sua linearità spazio temporale. In *2001* questo non accade. Kubrick lascia tutto avvolto nel mistero. Niente è esplicito. Non ci sono battaglie e scontri intergalattici, o meglio, ci sono ma hanno una natura del tutto interiore. Mancano inoltre dal film, come fa notare Michel Chion, tutte le battute del tipo: "attivate il fascio di iperenergia" oppure "stiamo entrando nel fascio di asteroidi"[2]. Dice Kubrick: "i film di fantascienza sono sempre stati sinonimo di mostri e sessualità, noi abbiamo cercato un altro approccio"[3]. Per la mancanza di tutto questo repertorio classico il film delude i ragazzi che lo vedono senza una giusta premessa. Nonostante ciò non sono pochi quelli che, dopo averlo visto e compreso, lo inseriscono nella lista dei preferiti.

Ma la natura oscura di quanto accade in *2001*, è proprio essa alla base della forza del film stesso. Per comprenderlo c'è bisogno di un approccio interpretativo. Nel corso di questi trent'anni le interpretazioni applicatevi sono state veramente tantissime. È il motivo anche del proliferare di così tanta letteratura attorno al fenomeno *2001*.

Di fronte a tanto materiale e a tante chiavi di lettura differenti ci chiediamo: c'è un punto di vista più meritevole rispetto agli altri? C'è un significato ultimo del film, che stia di diritto sopra a tutti e che in qualche modo coincida con le intenzioni del regista? La risposta è negativa. Kubrick stesso ha dichiarato più volte: "siete liberi di speculare sul significato filosofico e allegorico del film", e ancora: "ognuno è libero di speculare a suo gusto… io ho tentato di rappresentare un'esperienza

[1] Per brevità useremo sempre, nella presente lettura, la dicitura abbreviata *2001*.
[2] Cfr. Michel Chion, *Un'odissea del cinema. Il "2001" di Kubrick*, Lindau, Torino 2000, p. 18.
[3] *Ivi*, p. 36.

1

Figura 2.2: Corpo dell'opera

2.1.1 Generi Letterari

Come descritto da Jenna Moreci nel suo manuale dedicato alla scrittura creativa [4], il concetto di genere letterario si riferisce a una categoria di opere basata sullo stile e sul contenuto narrativo. Elementi come l'ambientazione, i personaggi, le creature fantastiche definiscono il genere di un'opera letteraria.

Tra i generi di narrativa più diffusi e riconosciuti si possono citare:

- **Rosa**: si concentra sulle relazioni sentimentali tra due o più personaggi.
- **Giallo**: ruota attorno a un crimine, spesso un omicidio, seguendo la narrazione dall'evento fino alla sua risoluzione.
- **Horror**: progettato per suscitare paura, disgusto o un senso di inquietudine nel lettore.
- **Fantasy**: caratterizzato da avventure che coinvolgono elementi magici, mondi immaginari o entrambi, spesso ispirati a tradizioni folkloristiche e mitologiche.
- **Fantascienza**: ambientato nel futuro o in contesti che prevedono tecnologie avanzate e scenari futuristici..

Questi generi rappresentano solo una parte della vasta gamma di categorie letterarie esistenti, che possono rendere la classificazione di un'opera complessa e talvolta ambigua. La presenza di numerosi sottogeneri contribuisce a questa complessità: molti generi sono infatti molto ampi e comprendono milioni di opere diverse. Ad esempio, il fantasy può includere storie di avventura, mondi immaginari e magia, ma al suo interno si distinguono sottogeneri come:

- **Fantasy epico**: caratterizzato da lunghe quest e da un'ampia costruzione del mondo narrativo;
- **Fantasy oscuro**: contraddistinto da tematiche violente e atmosfere cupe, spesso incentrato sul tema della morte;
- **Fantasy contemporaneo**: ambientato nel mondo reale, con elementi fantastici inseriti in un contesto quotidiano.

Per una corretta classificazione di un'opera è quindi fondamentale conoscere le caratteristiche e le regole dei generi e sottogeneri più rilevanti, al fine di restringere il campo e identificare la categoria più appropriata. Questo processo richiede un'attenta analisi delle sfumature narrative, evitando di attribuire eccessiva importanza a dettagli minori che, pur rilevanti per la trama, potrebbero non essere determinanti per la definizione del genere letterario.

Nel complesso, un libro, indipendentemente dal formato (cartaceo, digitale o audio), è considerato come un oggetto informativo e di intrattenimento, caratterizzato da una serie di attributi descrittivi che ne facilitano l'indicizzazione e la catalogazione.

2.2 Algoritmi di Raccomandazione

I sistemi di raccomandazione hanno lo scopo di filtrare un vasto insieme di elementi, in questo caso libri, per proporre all'utente quelli che potrebbero risultare di suo interesse [5]. Per raggiungere questo obiettivo, si utilizzano diverse tipologie di algoritmi, che però necessitano di informazioni aggiuntive, ovvero i cosiddetti *dati di interazione utente*. Questi dati includono valutazioni, recensioni, stato di lettura e cronologia d'uso, e costituiscono la base fondamentale per l'analisi comportamentale all'interno dei sistemi di raccomandazione.

Citando gli studi dell'ICAR [5]:*"Il successo di un metodo di raccomandazione dipende largamente dalla qualità e dalla completezza della rappresentazione degli interessi correnti dell'utente. In altri termini, perché le tecniche di predizione possano fornire raccomandazioni rilevanti e accurate è indispensabile disporre di modelli accurati."*

2.2.1 Importanza degli Algoritmi in numeri

Gli algoritmi di raccomandazione rivestono un ruolo cruciale nell'era digitale, poiché consentono di filtrare e selezionare contenuti rilevanti all'interno di cataloghi estremamente vasti e in continua crescita. Nel caso specifico dei libri, questi sistemi aiutano gli utenti a scoprire opere di interesse, migliorando l'esperienza di fruizione e aumentando l'efficacia delle piattaforme editoriali e di distribuzione [5].

A dimostrare che questi strumenti siano fondamentali nel settore digitale, Google, ad esempio, personalizza i risultati delle ricerche in base al profilo e alle interazioni dell'utente, migliorando la pertinenza delle informazioni fornite. Piattaforme di e-commerce come Amazon ed eBay utilizzano la profilazione per suggerire prodotti, contribuendo a generare una quota significativa del loro fatturato: si stima che Amazon ottenga circa il 35% delle vendite proprio grazie a questi algoritmi. Nel settore dell'intrattenimento, Netflix attribuisce ai sistemi di raccomandazione il 75% del consumo totale di contenuti sulla propria piattaforma[2].

[2]McKinsey `https://www.mckinsey.com/industries/retail/our-insights/how-retailers-can-keep-up-with-consumers`

Dal punto di vista economico, il mercato globale dell'intelligenza artificiale, che include anche gli algoritmi di raccomandazione, è in rapida espansione come mostrato nella Tabella 2.1.

In Italia, il settore dell'IA ha già raggiunto un valore di 674 milioni di euro nel 2023, con una crescita prevista del 34,8% per il 2024[3].

Tabella 2.1: Sintesi del mercato globale e italiano dell'Intelligenza Artificiale.

Aspetto	Valore e Periodo	Fonte
Mercato globale IA	\$244 miliardi (2025) → \$827 miliardi (2030). CAGR superiore al 24% (2025–2030).	[1]
Mercato AI generativa	\$54,4 miliardi (2025) → \$425,25 miliardi (2030). CAGR 50,87% (2025–2030).	[2]
Mercato IA Italia 2023	674 milioni di euro (+55% rispetto al 2022).	[3]
Crescita prevista IA Italia 2024	+34,8% (circa 909 milioni di euro).	[3]
Valore aggiornato IA Italia 2024	1,2 miliardi di euro (+58% rispetto al 2023).	[4]

***Note**: I dati sono tratti da report di società di analisi di mercato e osservatori specializzati.*

Fonti:

1. Statista: Artificial Intelligence Market Size Worldwide (2025–2030)
2. Precedence Research: Generative AI Market Size, Share, Growth Report (2025–2030)
3. Anitec-Assinform, NetConsulting cube: Il mercato dell'IA in Italia 2023
4. Osservatorio Artificial Intelligence, Politecnico di Milano: Mercato IA Italia 2024

Questa crescita è trainata dall'adozione sempre più diffusa di tecnologie AI in diversi ambiti, dal retail alla sanità, dalla finanza alla manifattura.

[3] ICOM `https://shorturl.at/yBDh6`

In conclusione, gli algoritmi di raccomandazione rappresentano non solo una tecnologia chiave per la gestione e la fruizione di contenuti digitali, ma anche un settore in forte espansione economica e strategica, che già influenza profondamente il modo in cui consumiamo informazioni, prodotti e servizi.

2.2.2 Cenni sulle Metodologie

Osservando la Figura 2.3 possiamo distinguere le principali famiglie di algoritmi utilizzati per la raccomandazione. Di seguito verranno descritti i loro pregi e difetti.

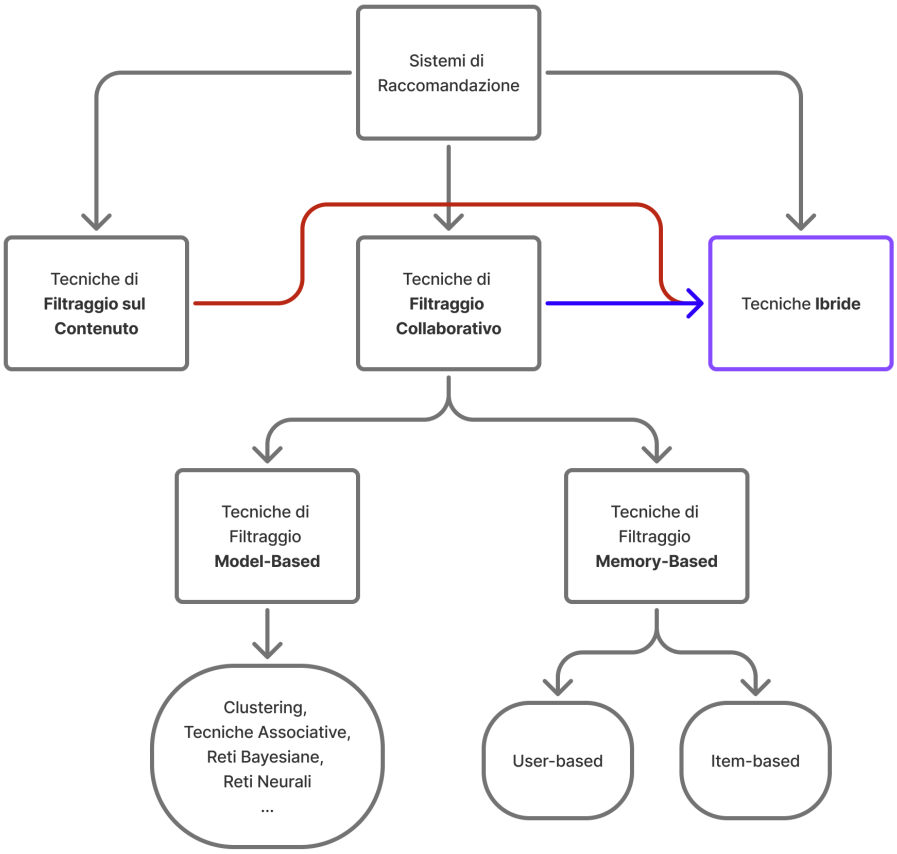


Figura 2.3: Visuale gerarchia di tecniche di raccomandazione.

Filtraggio sul Contenuto

Questo approccio si basa sulle caratteristiche degli item che un utente ha gradito in passato per raccomandargli item simili. Per un libro, le caratteristiche possono includere genere, argomenti trattati, stile di scrittura, autore, parole chiave, ecc. Viene costruito un profilo dell'utente basato sulle caratteristiche aggregate dei libri che

ha letto o valutato positivamente. Il sistema raccomanderà quindi nuovi libri le cui caratteristiche corrispondono maggiormente al profilo dell'utente[6, pp. 5–7,35–36,115,124].

Vantaggi

- Non soffre del problema del "cold-start" per i nuovi item, poiché può raccomandare un nuovo libro purché se ne conoscano le caratteristiche, indipendentemente dalle interazioni passate degli utenti.
- È efficace nel raccomandare item simili a quelli già noti e apprezzati dall'utente.

Svantaggi

- Richiede una descrizione dettagliata e strutturata degli item.
- Ha difficoltà nel raccomandare item che si discostano significativamente dai gusti pregressi dell'utente ("over-specialization").
- Soffre del problema del "cold-start" per i nuovi utenti, non avendo dati sufficienti per costruire un loro profilo.

Filtraggio Collaborativo Memory-Based

Il filtraggio collaborativo si basa sull'idea che le persone che hanno mostrato comportamenti simili in passato avranno probabilmente gusti simili in futuro. Questo approccio non necessita di una comprensione esplicita del contenuto degli item [6, pp. 8,22–26, 51, 64, 115]. Si divide principalmente in due sottocategorie:

- *Collaborative Filtering Basato sugli Utenti (User-Based CF)*: Trova utenti simili all'utente attivo (chi ha valutato item in modo simile) e raccomanda all'utente attivo gli item che gli utenti simili hanno gradito ma che l'utente attivo non ha ancora visto.
- *Collaborative Filtering Basato sugli Item (Item-Based CF)*: Identifica item simili a quelli che l'utente attivo ha gradito. La similarità tra item è calcolata in base ai pattern di valutazione degli utenti (due libri sono simili se sono stati valutati

in modo simile da molti utenti). Vengono quindi raccomandati item che sono simili a quelli già apprezzati dall'utente.

Vantaggi

- Non richiede metadati sugli item. È efficace nello scoprire nuovi interessi per l'utente, raccomandando item che sono popolari tra utenti con gusti simili ma che potrebbero essere diversi da ciò che l'utente ha letto finora.
- Gestisce bene preferenze complesse e soggettive.

Svantaggi

- Soffre del problema del "cold-start" sia per i nuovi utenti (pochi dati di interazione) che per i nuovi item (nessuna interazione).
- Richiede una notevole quantità di dati di interazione (problema della *"sparsity"*).
- Tende a raccomandare item popolari ("popularity bias") trascurando quelli di nicchia.

Filtraggio Collaborativo Model-Based

Utilizza modelli matematici/statistici (clustering, reti neurali, reti bayesiane) per prevedere le preferenze degli utenti.

Vantaggi

- Più scalabile rispetto agli approcci memory-based: adatto a grandi dataset.
- Può catturare pattern complessi nei dati.

Svantaggi

- Richiede tempo e risorse per l'addestramento dei modelli.
- Può soffrire di overfitting o generalizzare male se i dati sono scarsi o rumorosi.

Tecniche Ibride

Combinano più metodologie (tipicamente content-based e collaborative) per sfruttare i vantaggi di ciascuna e mitigare i rispettivi limiti.

Vantaggi

- Migliorano la qualità delle raccomandazioni, riducendo i problemi di "cold-start", "sparsity" e "popularity bias".
- Offrono maggiore flessibilità e robustezza, adattandosi a diversi scenari applicativi.

Svantaggi

- Maggiore complessità di implementazione e manutenzione.
- Richiedono più risorse computazionali e dati eterogenei.

2.3 Piattaforme di Raccomandazione Esistenti

Il panorama delle piattaforme online dedicate ai libri e alla lettura include diversi attori che offrono servizi di raccomandazione, sfruttando vari approcci algoritmici e modelli di interazione utente.

2.3.1 Goodreads

Goodreads, acquisita da Amazon nel 2013, è una piattaforma sociale per la lettura con oltre 85 milioni di visite mensili[4]. Il sistema di raccomandazione si basa principalmente su filtraggio collaborativo, un approccio introdotto da Resnick et al. (1994) che analizza pattern di valutazione e interazione tra utenti per generare suggerimenti[7]. La sua forza risiede nella vasta comunità e nel database di oltre 300 milioni di opere[5].

Tuttavia, la piattaforma presenta anche alcuni **limiti e criticità**.

[4]SimilarWeb `https://shorturl.at/BDxYu`

[5]Wikipedia `https://it.wikipedia.org/wiki/Goodreads`

Uno di questi è la possibilità che **le raccomandazioni includano libri o adattamenti non desiderati** dagli utenti, anche se il sistema prevede funzioni per escludere certi tipi di suggerimenti cliccando su *"not interested"*.

Inoltre, la **presenza di recensioni anonime** e la possibilità di manipolazioni come il "review bombing" possono influenzare negativamente la qualità delle valutazioni e quindi delle raccomandazioni.

Infine, alcuni utenti lamentano che le raccomandazioni a volte includano titoli già letti o valutati, riducendo l'efficacia del sistema.

Amazon, il gigante dell'e-commerce non ha ritenuto opportuno un investimento significativo che ne migliorasse le funzionalità. Trattandosi di un sito con un'infrastruttura obsoleta, infatti, i costi di revisione e di aggiornamento sarebbero stati molto elevati.[6]

2.3.2 TheStoryGraph

TheStoryGraph, lanciata nel 2020, ha registrato una crescita esponenziale, con oltre 3 milioni di utenti attivi mensili nel 2025[7]. Adotta un approccio innovativo basato su **metadati qualitativi** (emozioni, ritmo, complessità) e metadati strutturati. Questo modello, descritto in un'intervista al fondatore nel 2021, mira a superare le limitazioni dei sistemi basati su voti numerici[8]. La piattaforma utilizza tecniche di **content-based filtering** integrate con analisi sentimentale, un approccio supportato da studi sull'efficacia dei VARS *(value-aware recommender systems)*[8]

[6]Giornale della Libreria `https://shorturl.at/jRF96`

[7]SimilarWeb `https://shorturl.at/5xdya`

[8]Guardian `https://www.theguardian.com/books/2025/feb/16/goodreads-amazon-nadia-odunayo-the-storygraph`

2.4 Data Augmentation

Nel contesto dei sistemi di raccomandazione, la Data Augmentation si riferisce al processo di arricchire un dataset esistente con nuovi dati provenienti da fonti esterne. Questo processo è particolarmente critico per i sistemi di raccomandazione libraria, dove la costante pubblicazione di nuovi titoli rende rapidamente obsoleti i dataset statici.

La Data Augmentation non si limita semplicemente all'aggiunta di nuove righe di dati (ad esempio, nuovi libri nel catalogo). Piuttosto, implica un'attenta integrazione di nuove informazioni che possono arricchire gli attributi esistenti (es. aggiungendo tag di genere più specifici) o fornire nuovi attributi (es. informazioni sul tono emotivo del libro).

2.4.1 Obiettivi e Benefici

- **Aggiornamento Continuo**: Mantenere il dataset allineato con le ultime pubblicazioni, garantendo che il sistema di raccomandazione possa suggerire anche i titoli più recenti.
- **Arricchimento delle Informazioni**: Aggiungere dettagli più granulari e contestuali sui libri, come tag di genere più specifici, informazioni sul tono emotivo, o recensioni da diverse fonti.
- **Superare la Scarsità di Dati**: Integrare dati da diverse fonti per compensare la mancanza di informazioni in un dataset specifico, ad esempio, ottenendo descrizioni o recensioni da fonti esterne.
- **Migliorare la Precisione delle Raccomandazioni**: Fornire al sistema di raccomandazione una base di dati più ricca e accurata, consentendo di identificare meglio le preferenze degli utenti e di offrire suggerimenti più pertinenti.

2.4.2 Approcci

- **Integrazione di Fonti Esterne**: Acquisizione di dati da database librari, API di case editrici, siti di recensioni, e altre fonti.
- **Estrazione di Informazioni Testuali**: Utilizzo di tecniche di Natural Language Processing (NLP) per estrarre informazioni rilevanti dai riassunti dei libri, dalle recensioni, o da altri testi.
- **Creazione di Nuovi Attributi**: Combinazione di informazioni esistenti per generare nuovi attributi, ad esempio, calcolando un punteggio di "leggibilità" basato sul numero di pagine e sulla complessità del linguaggio.

2.4.3 Difficoltà

La *Data Augmentation* pone sfide importanti nei sistemi di raccomandazione, soprattutto per garantire **equità** e **completezza dei dati**. Come indicato nel *Recommender Systems Handbook* (pp. 688-689)[6], uno dei problemi principali è **ottenere dati adeguati per studiare la fairness di gruppo**, che richiede informazioni aggiuntive come genere o etnia — spesso difficili da raccogliere per **motivi legali, etici o di privacy**.

I dataset pubblici (es. Kaggle) sono utili per la **fairness individuale**, ma per quella di gruppo è necessario **integrare o generare nuovi dati**, ad esempio tramite inferenza statistica o dati sintetici. Tuttavia, queste pratiche **sono complesse** e **soggette a bias**.

L'integrazione di dataset pubblici o l'uso di annotazioni (professionali o via *crowdsourcing*) è possibile, ma spesso **costosa** e delicata, specie per quanto riguarda le **etichette e la privacy degli individui**. Tecniche automatiche come il riconoscimento di genere da nomi o immagini **sono sconsigliate** per problemi di accuratezza e di etica.

La **generazione di dati sintetici** può simulare attributi utili per valutare la fairness, ma **richiede validazione rigorosa** e una chiara comprensione dei limiti. È fondamentale **documentare in modo trasparente** tutte le scelte nella raccolta, integrazione e generazione dei dati per garantire riproducibilità e correttezza.

CAPITOLO 3

Applicazione Sviluppata

3.1 Contesto e Obiettivi del Progetto

Come evidenziato nella Sezione 1.2 la ricerca attuale presenta un gap significativo nell'affrontare la sfida della qualità dei dati da una prospettiva architetturale.

Obiettivo Primario: progettazione, implementazione e validazione di un'architettura software a componenti, che possa poi essere usata per migliorare la qualità delle raccomandazioni di libri attraverso un arricchimento dei metadati.

3.2 Metodologia e Fasi di di Sviluppo

Lo sviluppo del progetto è stato guidato da una metodologia iterativa, supportata da un robusto stack tecnologico scelto per affrontare le specifiche sfide di un progetto data-intensive. L'intero processo è stato tracciato e gestito tramite GitHub, garantendo trasparenza, riproducibilità e una cronologia chiara dell'evoluzione del progetto.

- **Repository Centrale**: GitHub è stato utilizzato come repository centrale per il codice sorgente. Questo ha garantito un backup costante, ha permesso di tracciare ogni modifica e funge da portfolio pubblico del lavoro svolto.

- **Branching Strategy Semplificata**: È stata adottata una strategia di branching leggera. Il branch main è stato mantenuto come la versione stabile del codice, mentre lo sviluppo di nuove funzionalità (come il motore di raccomandazione o il web server) è avvenuto su branch dedicati (es. feature/recommendation-engine). Questo ha permesso di sperimentare e sviluppare senza compromettere la stabilità della base di codice principale, per poi integrare la funzionalità completata tramite Merge.

3.2.1 Fasi Individuate

Il progetto è stato suddiviso nelle seguenti fasi sequenziali, che riflettono la struttura del sistema:

1. Analisi del Dataset e Pre-elaborazione (ETL).
2. Sviluppo del Sistema di Inizializzazione (Core).
3. Sviluppo del Sistema di Data Augmentation.
4. Sviluppo del Sistema di Raccomandazione.
5. Sviluppo di una Webapp minimale.
6. Analisi delle Performance del sistema.

3.2.2 Tecnologie Impiegate

- **Linguaggio di Programmazione: Python** è lo standard per progetti che coinvolgono analisi dei dati e machine-learning.
- **Database: MongoDB** essendo un database NoSQL orientato ai documenti, si è rivelato la scelta ideale per ospitare i dati grezzi provenienti da file JSON. La sua natura schema-less ha permesso una grande flessibilità durante la fase iniziale di ingestione ed esplorazione, mentre le sue potenti capacità di aggregazione e indicizzazione (inclusa la ricerca testuale) sono state fondamentali per le query complesse e le performance dell'applicazione finale. MongoDB supporta lo

sharding, ovvero la distribuzione del database su più macchine, garantendo scalabilità orizzontale e preparandolo a futuri sviluppi su larga scala.

- **Motore di raccomandazione**:
  - *Annoy* (Approximate Nearest Neighbors Oh Yeah), sviluppato da Spotify, per la ricerca approssimata dei vicini in spazi ad alta dimensionalità, ottimizzato per velocità e basso consumo di memoria.
  - *FAISS* (Facebook AI Similarity Search), libreria ottimizzata per il calcolo veloce delle similarità su grandi dataset, utilizzata per indicizzazione e ricerca vettoriale ad alte prestazioni.
- **Modelli linguistici e NLP**:
  - Utilizzo di *Sentence Transformers* e *Transformers* per il calcolo degli embedding testuali e l'analisi semantica delle descrizioni dei libri.
  - È stato impiegato *Ollama* per eseguire in locale il modello **Gemma-3**, utilizzato nella pipeline di normalizzazione e arricchimento dei dati.
- **Goodreads Metadata Scraper**: progetto open source[1] utilizzato per il recupero di metadati aggiuntivi e per la validazione del sistema di data augmentation. L'obiettivo è stato ottenere informazioni 'originali' ed attuali e confrontare l'approccio proposto con differenti provider conformi alle normative vigenti.
- **Web Framework: Flask**: per esporre la funzionalità di ricerca e raccomandazione attraverso un'interfaccia web. La sua natura 'micro' permette di creare applicativi leggeri e performanti, senza overhead di un framework più complesso.

[1] `https://github.com/tiago-cos/goodreads-metadata-scraper`

3.3 Acquisizione del Dataset

Il primo passo fondamentale del processo di ricerca è consistito nell'identificazione e selezione di un dataset adeguato. La scelta è ricaduta sull'esaustivo dataset di Goodreads, curato e reso disponibile dalla University of California San Diego (UCSD) per scopi accademici[9; 10]. Questo insieme di dati è stato ritenuto ideale per diversi motivi:

- **Origine Accademica**: la sua provenienza garantisce un elevato standard di raccolta e documentazione.
- **Ricchezza**: le dimensioni del dataset offrono una base dati statisticamente significativa per l'addestramento e la validazione di algoritmi complessi.
- **Autenticità**: essendo basato su interazioni reali e non su dati sintetici, permette di sviluppare un sistema che affronta le sfide del mondo reale, come la sparsità dei dati e la varietà del catalogo.

Il dataset si articola in due macro-categorie principali: metadati relativi alle opere letterarie e dati di interazione tra utenti e libri. Nello specifico, l'insieme di dati grezzi comprende:

- **Opere Letterarie**: informazioni su 2.36 milioni di pubblicazioni, ovvero 1.52 milioni di opere uniche.
- **Entità Correlate**: dati su 829.529 autori e 400.390 serie di libri.
- **Interazioni Utente**: oltre 228 milioni di interazioni da parte di 876.145 utenti

Offre quindi un'opportunità unica per esplorare in profondità il mondo dei libri e delle loro comunità di lettori, analizzando tendenze, preferenze e le dinamiche di interazione che caratterizzano la piattaforma, con dati non sintetici.

3.3.1 Pre-elaborazione e Ottimizzazione dei Dati (ETL)

Il dataset ha pertanto dimensioni imponenti ed una dimensione totale di circa 29GB, distribuita in file JSON per garantirne la sua trasportabilità.

Si è quindi reso necessario un processo metodologico di **Extract, Transform, Load (ETL)** per trasformare i dati grezzi in asset strutturati, performanti e pronti per l'analisi.

La strategia di ottimizzazione è stata guidata da tre principi chiave:

1. **Cernita dei Campi (Field Trimming)**: analisi preliminare per identificare campi non essenziali per il progetto.
2. **Sfruttamento di Formati di Archiviazione Efficienti**: la migrazione al database MongoDB con la sua tecnica di compressione dati BSON (Binary JSON) ha ridotto significativamente lo spazio occupato, in quanto non ripete le chiavi testuali per ogni documento.
3. **Indicizzazione Strategica**: è stato fatto un investimento strategico in termini di spazio per la creazione di indici su campi più interrogati. Questa scelta è un trade-off dove si sacrifica spazio su disco per ottenere una drastica riduzione dei tempi di risposta delle query.

L'applicazione della strategia descritta ha portato a una riduzione complessiva dello spazio occupato di circa il 34%, passando da 29 GB a un totale di circa 19 GB (13,5 GB per i dati e 5,5 GB per gli indici) come mostrato in Tabella 3.1.

Collezione	Sorgente (JSON)	Dati (DB)	Indici (DB)	Totale (DB)
reviews	15.6 GiB	9.98 GB	617.62 MB	**~10.6 GB**
books	8.6 GiB	2.47 GB	4.82 GB	**~7.3 GB**
original_books	697.5 MiB	183.35 MB	16.54 MB	**~200 MB**
authors	101.0 MiB	40.30 MB	34.80 MB	**~75 MB**
book_series	105.9 MiB	54.53 MB	4.14 MB	**~59 MB**
book_genres	190.6 MiB	88.20 MB	57.36 MB	**~145 MB**

Tabella 3.1: Confronto tra dimensioni dei file JSON e corrispondenti collezioni nel database MongoDB.

Field Trimming

Sebbene i dataset di partenza fossero estremamente ricchi di informazioni, non tutti i campi erano pertinenti agli obiettivi del sistema di raccomandazione. Alcuni erano ridondanti, altri specifici della piattaforma Goodreads. L'obiettivo di questa fase era ridurre l'ingombro.

Collezione *books* Questa è stata la collezione che ha beneficiato maggiormente del processo di trimming, i dati sono stati ridotti del **73%** come evidenziato dalla Tabella 3.1. I campi omessi e le relative motivazioni sono i seguenti:

- **Identificatori di Piattaforma**: campi come *ISBN*, *ISBN13*, *kindle_asin*... sono stati scartati. Sebbene utili in contesti di vendita al dettaglio, ho optato di standardizzare l'identificazione delle opere con gli ID interni *book_id* e *work_id*.
- **URL e Metadati Esterni**: *link*, *url*, *image_url* sono stati rimossi in quanto puntavano a risorse esterne non necessarie per il core del motore di raccomandazione, contribuendo al risparmio di spazio.

- **Informazioni sull'Edizione**: campi come *format*, *country_code*, *language_code* sono stati esclusi per concentrare l'analisi sull'opera letteraria intrinseca piuttosto che sulle sue singole manifestazioni editoriali.

Collezione *authors*

- Dalla collezione **authors**, è stato rimosso il campo *average_rating*, preferendo mantenere i dati grezzi come *ratings_count* e *text_reviews_count*, che sono indicatori più diretti della popolarità e attività di un autore.

Collezione *original_books*

- Sono stati eliminati metadati interni di Goodreads (*default_description_language_code*, *best_book_id*, etc.) e il campo *media_type*, ritenuti non pertinenti per il nostro scopo.

Collezione *reviews*

- Questa è la collezione più grande e la riduzione è stata del 37%. Sono stati mantenuti tutti i campi originali. La riduzione è merito dell'alta comprimibilità dei testi delle recensioni nel campo *review_text*.

Collezione *book_genres*

- Non sono stati scartati campi, con una riduzione del 25%.

Collezione *book_series*

- Eliminati i campi *numbered* e *note*, 47% di spazio risparmiato.

3.3.2 Pulizia e Preparazione Dati

1. **Gestione della Ridondanza delle Edizioni**: il problema più critico del dataset era la frammentazione dei dati di una singola opera su più edizioni. Un libro famoso poteva avere decine di voci separate, ognuna con le proprie recensioni e valutazioni, rendendo le query di aggregazione molto più lente.

    - **Strategia**: per ogni gruppo di libri appartenenti alla stessa opera, è stata designata come 'edizione primaria' quella con il maggior numero di recensioni, utilizzandola come proxy della versione più popolare e discussa. Tutte le edizioni 'secondarie' sono state scartate dalla collezione principale *books* e le recensioni sono state migrate ed associate all'edizione primaria.

2. **Eliminazione dei Generi Orfani**: l'operazione di pulizia precedente ha generato un effetto collaterale, la collezione *book_genres* conteneva riferimenti a *book_id* di edizioni eliminate.

    - **Strategia**: Identificati i record orfani, sono stati eliminati dal database.
    - **Risultato**: la coerenza tra le collezioni è stata ripristinata, garantendo che ogni informazione sui generi si riferisse a un libro effettivamente presente nel sistema.

3. **Selezione dei Tag Utente per l'Estrazione dei Generi**: il dataset non permette una perfetta comprensione dei generi dei libri, in quanto sono stati etichettati per 'macro-aree'.

    - **Analisi e Frequenza** sono stati aggregati e contati tutti i tag unici per capire quali fossero i più utilizzati.
    - **Filtraggio**: Dopo aver scartato tag non informativi (come '*to-read*', '*owned*', '*favorites*') manualmente, i tag più frequenti corrispondevano in parte a generi letterari consolidati, sottogeneri, temi o tropi narrativi (*cyberpunk*, *dystopian*, *noir*...). Da oltre un milione di tag è stata effettuata una cernita per selezionare una lista curata di circa 1500 tag rilevanti.
    - **Considerazioni**: Questi vettori non definiscono un genere in modo rigido, ma rappresentano la similarità di contenuto e tema tra i libri. È proprio

questa impronta che alimenta il motore di raccomandazione content-based: il sistema suggerisce nuovi libri trovando quelli la cui impronta digitale è più simile a quella dei libri che un utente ha già amato.

In sintesi, questa meticolosa fase di pulizia non è stata un semplice passaggio preliminare, ma una componente fondamentale del progetto che ha garantito l'integrità, la coerenza e l'affidabilità dei dati, rendendo possibili le successive, complesse analisi di profilazione e raccomandazione.

3.3.3 Analisi del Dataset

Dopo un'attenta pulizia (vedi Sezione 3.3.2) ed aggregazione dei dati, l'analisi del dataset rivela caratteristiche di una community di lettori matura ed altamente coinvolta. In media, ciascun utente lascia **33,83 recensioni**, a testimonianza di un elevato livello di partecipazione e interazione all'interno della piattaforma.

Il grafico 3.1 mostra una crescita esponenziale del numero di libri pubblicati a partire dai primi anni 2000 per poi arrestarsi dopo il 2010 fino al raggiungimento del cut-off del dataset.

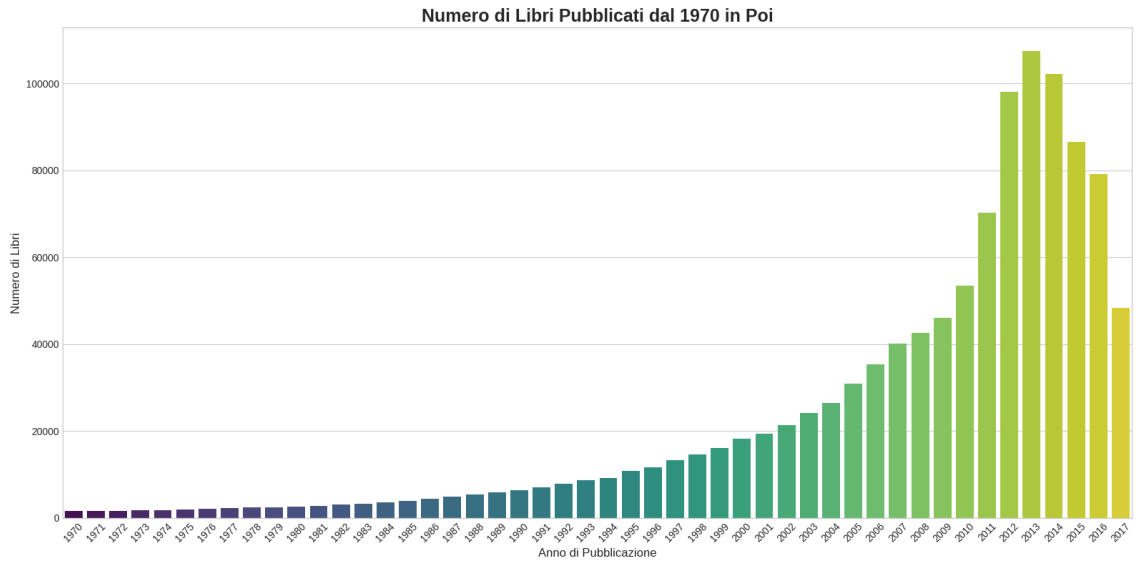


Figura 3.1: Distribuzione delle pubblicazioni negli anni.

Interpretazione: Questo non riflette un aumento della produzione editoriale tradizionale, ma piuttosto è una scelta strategica di Goodreads, che preferisce includere nel proprio catalogo i libri più recenti e facilmente reperibili dai lettori.

Il secondo grafico 3.2 svela la composizione del catalogo, confermando le tendenze di mercato e rivelando una caratteristica fondamentale dei dati:

- La *Fiction* è di gran lunga la categoria più vasta, seguita da generi 'pilastro' come *Non-Fiction*, *History* e *Romance*.
- La statistica sui **Libri Multi-Genere** è estremamente significativa. Quasi la metà dei libri è etichettata con più di un genere, indicando che i confini tra le categorie sono fluidi e spesso sovrapposti. Un libro *Fantasy, Paranormal* è frequentemente anche *Young-Adult* e *Romance*.

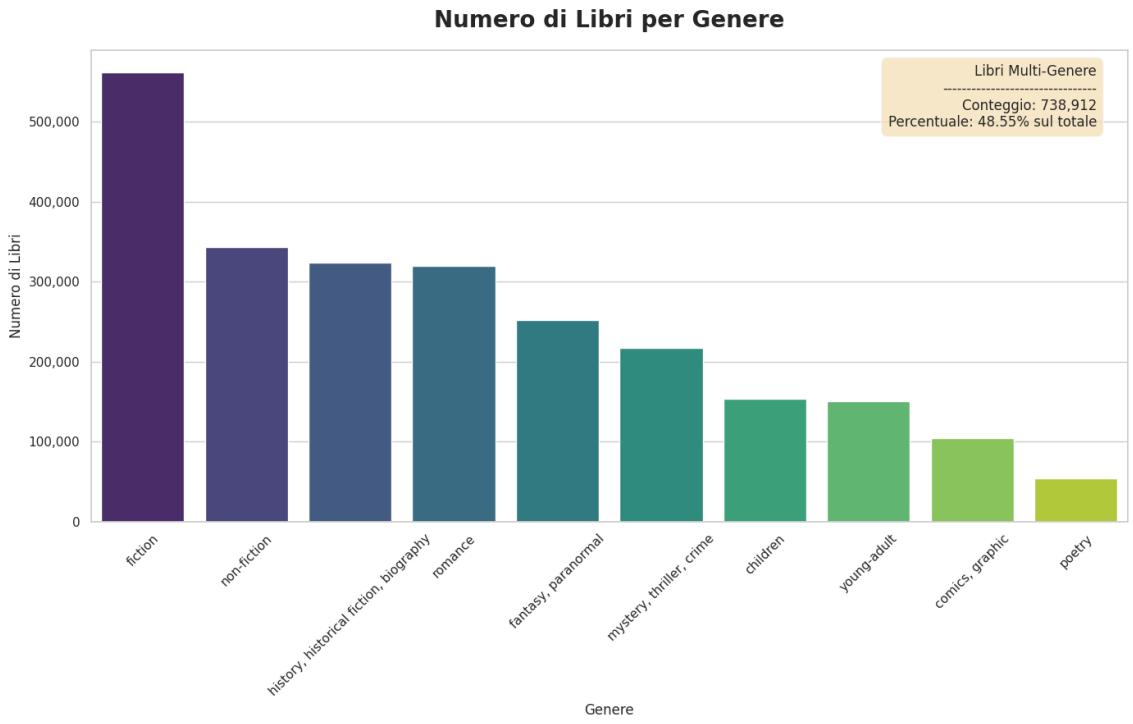


Figura 3.2: Distribuzione dei generi.

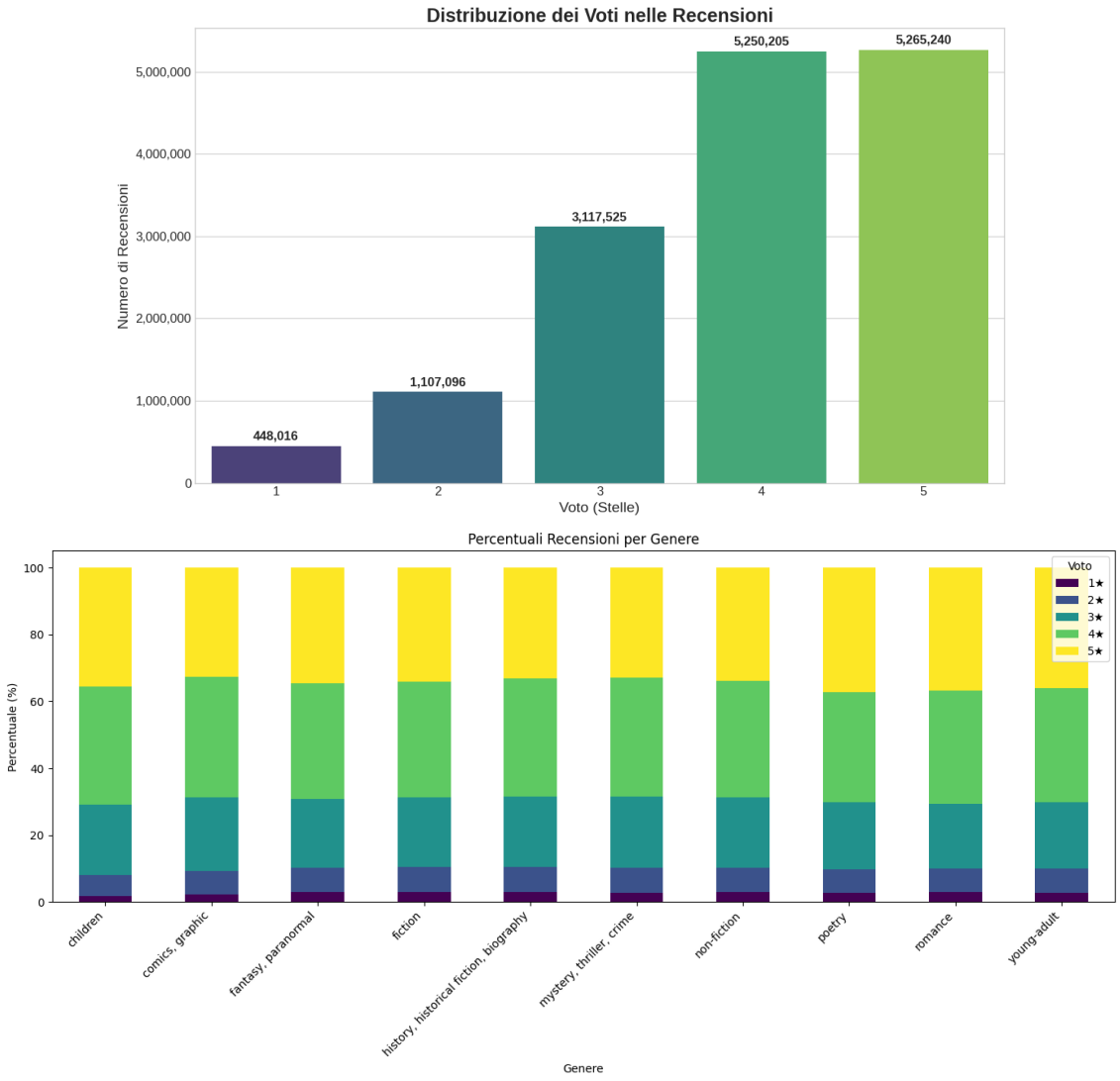


Figura 3.3: Distribuzione stelle nelle recensioni

I due grafici in Figura 3.3 letti insieme raccontano una storia coerente sul comportamento degli utenti:

- **Distribuzione Assoluta**: i voti sono pesantemente sbilanciati verso l'alto. I voti a 4 e 5 stelle costituiscono la stragrande maggioranza, surclassando i voti bassi.
- **Distribuzione Percentuale**: il grafico delle recensioni per genere conferma che questa tendenza è **universale** per tutti i generi.

Interpretazione: il fenomeno è spiegabile da molteplici motivazioni:

- **Bias di Autoselezione**: gli utenti tendono a leggere e recensire libri che già pensano di apprezzare.
- **Effetto 'DNF'**: chi abbandona un libro per scarso gradimento raramente lo recensisce, eliminando così potenziali voti negativi (Did Not Finish).
- **Cultura della Community**: la piattaforma Goodreads ha favorito la formazione di una community non particolarmente tossica.

Dati Mancanti

Nell'analisi del dataset è stata riscontrata una significativa quantità di dati mancanti, in particolare riguardo alle descrizioni e al numero di pagine ('page_count'). La tabella 3.2 riassume la distribuzione dei casi osservati.

Tabella 3.2: Distribuzione dei dati mancanti nel dataset

Categoria	Occorrenze
Libri senza descrizione ma con `page_count`	348.924
Libri senza `page_count` ma con descrizione	260.810
Libri senza descrizione e senza `page_count`	227.358
Libri con data errata	896

Ultime Considerazioni

- La principale sfida per il sistema di raccomandazione non consiste tanto nell'escludere i libri di scarsa qualità, quanto nel saper distinguere tra un libro "buono" (4 stelle) e un libro "eccezionale" (5 stelle), che l'utente apprezzerà realmente.

- Questo dato conferma l'importanza di adottare un profiler utente ponderato: un sistema di raccomandazione basato esclusivamente su un singolo genere per libro risulterebbe inefficace, poiché non coglierebbe le sfumature e la complessità dei gusti individuali.

- Analizzando i tag manualmente assegnati dagli utenti ai libri nella collezione *books*, tramite l'attributo *popular_shelves*, si riscontra la presenza di oltre un milione di tag distinti. Si osserva inoltre che i tag più frequenti spesso rappresentano generi letterari più specifici e dettagliati rispetto a quelli presenti nella collezione *book_genres*.

3.4 Sistema di Inizializzazione (Core)

Il cuore operativo del sistema è rappresentato dal sottosistema di inizializzazione, denominato "Core". Progettato per essere agnostico rispetto alle specifiche logiche di business (come l'ETL o la raccomandazione), questo insieme di moduli incapsula la logica fondamentale per l'avvio, la configurazione e la gestione dell'intera applicazione. La sua funzione è quella di fornire una base solida e coerente su cui gli altri sottosistemi possono operare in modo affidabile, garantendo manutenibilità, robustezza e flessibilità.

3.4.1 Scelte Architetturali

La progettazione del Core è stata guidata da principi consolidati di ingegneria del software, scelti per soddisfare i requisiti di estensibilità e manutenibilità, che sono centrali in questa tesi.

- **Separazione delle Responsabilità**: Ogni modulo del Core ha una responsabilità unica e ben definita. Questo principio è particolarmente evidente nella gestione dell'interfaccia a riga di comando (CLI), suddivisa in tre moduli distinti: `argument_definer.py` è responsabile unicamente della definizione dei comandi, `argument_dispatcher.py` si occupa del loro instradamento (routing), e `dispatcher_actions.py` contiene la logica effettiva per la loro esecuzione. Questa separazione rende il codice più pulito, più facile da testare e più semplice da estendere con nuovi comandi.

- **Pattern Singleton**: Le classi `LoggerManager` e `PathRegistry` implementano il pattern Singleton. Questo garantisce un unico punto di accesso globale a risorse critiche come il logger e il registro dei percorsi, evitando la creazione di istanze multiple e incoerenti. La gestione centralizzata previene la duplicazione di codice e assicura che ogni parte del sistema faccia riferimento alla stessa configurazione di logging e alla stessa mappatura dei percorsi.

- **Configurazione Centralizzata**: Per evitare l'hardcoding di parametri (come stringhe di connessione al database, percorsi di file o livelli di log), tutti i valori

operativi sono esternalizzati in un file `config.json`.
Il modulo `app_config_loader.py` è l'unico responsabile del caricamento, della validazione e della fornitura di questi parametri, rendendo l'applicazione facilmente adattabile a diversi ambienti (sviluppo, test, produzione) senza alcuna modifica al codice sorgente.

3.4.2 Flusso di Inizializzazione

L'avvio dell'applicazione segue una sequenza rigorosa e ben orchestrata dal modulo `core/app_initializer.py`, che assicura che tutti i servizi siano pronti prima che venga eseguita qualsiasi logica di business.

1. **Punto di Ingresso (`run.py`)**: Il suo ruolo è minimale: istanzia il `LoggerManager` per attivare un logging di base immediato, inizializza il `PathRegistry` con il percorso radice del progetto e delega l'intero processo di inizializzazione alla funzione `initialize_app`.

2. **Parsing Preliminare della Configurazione**: `app_initializer.py` esegue una prima scansione degli argomenti della CLI per il solo flag –config. Questa scelta progettuale strategica permette all'utente di specificare un file di configurazione personalizzato, che deve essere caricato prima di ogni altra operazione.

3. **Caricamento della Configurazione**: Viene invocato `app_config_loader.py` per caricare il file config.json (quello di default o quello specificato dall'utente). Se il file non viene trovato, il modulo è in grado di crearne uno di default, garantendo che l'applicazione possa sempre avviarsi.

4. **Finalizzazione del Logger**: `LoggerManager.setup_logger` viene chiamata per configurare il logger principale dell'applicazione con il livello, il formato e il file di output specificati. Questo logger sostituisce quello di pre-inizializzazione, che a quel punto viene disattivato per prevenire output duplicati.

5. **Registrazione dei Percorsi Dati**: La sezione *data_paths* del file di configurazione viene letta e utilizzata per popolare il `PathRegistry`, creando contestualmente le directory necessarie sul filesystem.

6. **Parsing Completo e Dispatching dei Comandi**: Infine, viene istanziato l' `ArgumentDefiner` per definire l'intera grammatica della CLI. Gli argomenti vengono parsati completamente e passati all' `ArgumentDispatcher`, che si occupa di instradare la richiesta alla funzione di esecuzione appropriata in `dispatcher_actions.py`, implementando la logica di business per ciascuna azione via CLI, interagendo con gli altri moduli dell'applicazione.

In sintesi, il sottosistema Core non è un mero script di avvio, ma una base architetturale capace di fornire un ambiente di esecuzione robusto, configurabile e facilmente manutenibile ottenuta attraverso l'applicazione dei principi di programmazione SOLID.

L'integrazione di tutti gli altri sottosistemi più complessi dipendenti da esso sono presentati successivamente.

3.4.3 Il Ruolo del Core nel Processo di Data Ingestion

Analizzata l'architettura del Core, è fondamentale illustrare come questa abiliti e supporti il processo di ingestione dei dati nel database, una fase cruciale derivata dall'analisi del dataset (Sezione 3.3.1). Sebbene il processo ETL venga eseguito una tantum in fase di setup, è stato progettato seguendo gli stessi principi di modularità e configurabilità del resto del sistema, affidandosi interamente ai servizi forniti dal Core. Il flusso operativo può essere scomposto come segue:

1. **Invocazione e Configurazione Iniziale**: Il processo viene attivato dall'utente tramite un comando specifico della CLI, *'etl –load-all'*. Il modulo ArgumentDispatcher intercetta la richiesta e invoca la funzione di esecuzione corrispondente e passandole due istanze singleton gestite dal Core: la configurazione applicativa completa e il PathRegistry.

   - **Connessione al Database**: Il primo componente attivato è `MongoDBConnection` che interroga `PathRegistry` per recuperare l'URI e il nome del database. Questo disaccoppia completamente la logica di connessione dalla struttura del filesystem.
   - **Localizzazione dei Dati Sorgente**: Quando il motore ETL deve leggere un file di dati (es. goodreads_book_authors.json), non utilizza percorsi relativi, ma interroga dinamicamente il `PathRegistry` (registry.get_path('raw_datasets_dir'). Questa dipendenza dal Core rende il processo di ingestione estremamente portabile e riconfigurabile.

2. **Motore ETL (`etl/loader.py`)** questo modulo contiene la logica per estrarre e caricare i dati. È progettato per gestire file di grandi dimensioni in modo efficiente.

   Fase 1: Extract

   Supporto Multi-formato: Gestisce sia file CSV che JSON. La funzione `load_json_items` è particolarmente flessibile, in quanto è in grado di processare sia file JSON standard (un unico array di oggetti) sia file NDJSON, che sono molto comuni nei dataset di grandi dimensioni.

Uso di Generatori: Le funzioni *load_csv_items* e *load_json_items* non caricano l'intero file in memoria. Invece, usano yield per restituire un record alla volta. Permettendo di processare file da svariati gigabyte senza problemi di performance.

Fase 2: Transform

Mapping Configurabile: La logica di trasformazione è definita in file di configurazione ETL separati. Questi file specificano quale campo del file sorgente (*src_field*) deve essere mappato a quale campo nel database (*dst_field*) e a quale tipo di dato (*type*). Un esempio per una sola *collection* è fornito in basso 3.1.

Listing 3.1: Esempio di configurazione ETL per autori

```
{
  "file": "goodreads_book_authors.json",
  "collection": "authors",
  "mapping": {
    "src_field": {"field": "dst_field", "type": "str"},
    ...
```

Conversione e Pulizia dei Dati: La funzione `convert_type` si occupa di convertire i valori nel tipo corretto, senza di esso tutti i campi verrebbero trattati come stringhe.

Fase 3: Load

Chunking: Invece di inserire i record uno per uno, il loader li raggruppa in 'chunk' di dimensione configurabile ed utilizza il metodo `insert_many` di PyMongo, inviando un lotto di documenti al database in una singola operazione, riducendo drasticamente i tempi di caricamento.

3.5 Sistema di Data Augmentation

Per superare i limiti dei dataset statici e arricchire le informazioni sui libri, è stato progettato e implementato un sottosistema di Data Augmentation. L'obiettivo di questo componente è aggiungere ed aggiornare libri al catalogo, attingendo dinamicamente da una pluralità di fonti esterne. Questo processo è fondamentale per migliorare la qualità dei dati, che a sua volta è un prerequisito per generare raccomandazioni più accurate e pertinenti.

Le scelte progettuali sono state guidate dall'obiettivo di realizzare un sistema flessibile, capace di integrare strumenti esterni interrogabili da riga di comando per l'arricchimento dei metadati. Tra i numerosi tool disponibili, come quelli utilizzati da applicazioni quali *Calibre* per la gestione di librerie di eBook, sono stati presi in considerazione anche diversi progetti open source presenti su GitHub. In particolare, è stato adattato e integrato il progetto *Goodreads Metadata Scraper*[2], che rappresenta un esempio concreto di come l'architettura sviluppata consenta di includere provider esterni, superando le limitazioni dovute alla chiusura delle API ufficiali di Goodreads.

3.5.1 Architettura a Provider

Il cuore del sistema di arricchimento dati è basato sul Strategy Pattern. La logica per interrogare una specifica fonte dati (es. API di Google Books, OpenLibrary, uno script CLI locale) è incapsulata all'interno di classi concrete, denominate "Provider".

Tutti i provider implementano una singola interfaccia comune, *BookDataProvider* (definita in book_data_provider_interface.py). Questa interfaccia impone un contratto chiaro: ogni provider deve essere in grado di:

- Identificarsi tramite un nome (get_name()).
- Accettare una richiesta di ricerca per un libro (fetch_data()) e restituire i metadati trovati in un formato standardizzato, definito dalla *TypedDict BookMetadata*.

Questo approccio offre un'eccezionale **estensibilità**: per aggiungere il supporto a una nuova fonte dati, è sufficiente creare una nuova classe che implementi l'interfaccia BookDataProvider, senza dover modificare minimamente il resto del sistema.

[2] `https://github.com/tiago-cos/goodreads-metadata-scraper`

I provider concreti implementati includono:

- **OpenLibraryProvider** e **GoogleBooksProvider**: Esempi classici di provider che interagiscono con API web RESTful. Gestiscono la costruzione delle query, l'invio delle richieste HTTP, la gestione degli errori e la parsificazione delle risposte JSON.
- **CliBookProvider**: Un provider particolarmente innovativo che agisce da "wrapper" per qualsiasi strumento a riga di comando. È stato progettato per essere altamente generico

3.5.2 Aggregazione dei Metadati

La gestione dei diversi provider è affidata alla classe `MetadataAggregator`.

Il suo compito è unicamente quello di consolidare le informazioni restituite, rispettando un ordine basato sulla completezza e affidabilità dei dati forniti: i provider che restituiscono metadati più completi vengono interrogati per primi, seguiti da quelli con informazioni parziali.

Il processo di aggregazione è intelligente e ottimizzato:

1. **Consolidamento Progressivo**: L'aggregatore mantiene un singolo oggetto BookMetadata che viene progressivamente arricchito. Quando un provider restituisce un dato (es. il numero di pagine), questo viene aggiunto all'oggetto solo se quel campo era mancante.
2. **Interruzione Anticipata (Early Stopping)**: Se tutti i campi chiave (es. descrizione, generi, numero di pagine) vengono popolati prima di aver interrogato tutti i provider, il processo si interrompe, evitando chiamate API superflue e migliorando le performance.
3. **Tracciamento delle Fonti (Data Provenance)**: Per ogni campo aggiunto, l'aggregatore tiene traccia del provider che lo ha fornito. Questa informazione è preziosa per il debugging e per valutare l'affidabilità delle diverse fonti dati.

3.5.3 Logica di Business e Database

Mentre i provider e l'aggregatore si occupano del "come" recuperare i dati, la logica di business del "cosa" fare con essi è incapsulata nei ***Service Layers*** (`BookCreationService` e `BookUpdateService`).

- **`MongoBookRepository`** (Pattern Repository): Questo componente astrae completamente l'interazione con il database MongoDB. Tutte le query (creazione, lettura, aggiornamento) sono centralizzate in questa classe. I servizi non "conoscono" la struttura delle collection di MongoDB, ma comunicano con il repository tramite metodi ben definiti. Questo disaccoppiamento è cruciale per la manutenibilità: se in futuro si decidesse di migrare a un altro tipo di database, solo il repository dovrebbe essere riscritto.

- **`BookCreationService`**: Gestisce il flusso di lavoro per l'aggiunta di un nuovo libro. Orchestra le chiamate: prima verifica se il libro esiste già (tramite il repository), poi invoca l'aggregatore per recuperare i metadati e infine, se i dati sono sufficienti, li formatta e li passa al repository per l'inserimento nel database.

3.5.4 Mapping dei Generi

Un problema centrale nell'uso di dati provenienti da fonti diverse (specialmente quelle basate su tag utente, o "folksonomie") è l'inconsistenza dei generi. Nel dataset di partenza sono definiti 10 'macro-generi'. È stato quindi necessario normalizzare i generi dei nuovi libri.

Per affrontare questa sfida, è stata implementata una mappatura flessibile.

- **Approccio su Parole Chiave**: Il primo metodo, `map_scraped_genres_to_predefined`, utilizza un approccio basato su regole e parole chiave. Una lista di generi in formato libero viene confrontata con un dizionario predefinito (GENRE_KEYWORD_MAP) per associarla a un insieme limitato e coerente di categorie (una "tassonomia").

- **Approccio Basato su LLM**: Per una mappatura più sofisticata e sensibile al contesto, è stata implementata la classe `LLMGenreMapper`. Per questa tesi è stato designato Gemma3 con 12 miliardi di parametri, un Large Language Model (LLM) eseguito in locale (tramite Ollama) che analizzando non solo i generi, ma anche il titolo e la descrizione del libro, fornisce risultati aventi le categorie più appropriate. Il prompt inviato al modello è stato attentamente ingegnerizzato (prompt engineering) per fornire istruzioni precise e richiedere un output in formato JSON strutturato.

Scelta Strategica dell'LLM in Zero-Shot Learning

La decisione di utilizzare un Large Language Model (LLM) pre-addestrato come Gemma-3, piuttosto che addestrare un modello di classificazione tradizionale o effettuare il fine-tuning del LLM stesso, non è stata casuale:

Addestrare da zero un modello di classificazione classico avrebbe introdotto tre ostacoli significativi, che l'approccio LLM supera nativamente:

- **Focalizzazione del Progetto**: L'obiettivo primario di questa tesi non era la creazione di un nuovo classificatore di generi, ma la progettazione e l'implementazione di un'architettura di arricchimento dati che fosse **robusta, estensibile**

e manutenibile. Iniziare un ulteriore ciclo di sviluppo per un classificatore avrebbe deviato risorse significative dall'obiettivo principale.

- **Time-to-Value**: L'adozione di un LLM pre-addestrato ha permesso di ottenere una soluzione di alta qualità in una frazione del tempo. Lo sviluppo si è concentrato sul prompt engineering, un processo iterativo rapido che ha richiesto poche ore, non settimane. Questo approccio ha permesso di integrare una logica di categorizzazione sofisticata nel sistema in tempi brevi, dimostrando il valore dell'architettura complessiva senza rimanere bloccati in un lungo ciclo di machine learning.
- **Agilità e Manutenibilità**: Se in futuro la tassonomia dei generi dovesse cambiare, con il nostro sistema, è sufficiente aggiornare la lista di generi predefiniti all'interno del prompt, un'operazione quasi istantanea.

Anche l'opzione del fine-tuning è stata scartata a favore del prompt engineering:

- **Assenza di Hardware**: Il fine-tuning è un processo computazionalmente intensivo che richiede hardware specializzato ed ulteriori competenze specifiche.
- **Performance nei Task**: Come evidenziato da analisi di settore[11] e benchmark tecnici[12], i modelli della famiglia Gemma sono particolarmente performanti in task di instruction-following come la classificazione di documenti[13], dove la comprensione del linguaggio e l'aderenza a regole precise sono più importanti del ragionamento multi-passo complesso.

Il sistema adotta un doppio meccanismo di controllo (a due strati) per garantire che l'LLM non possa inserire generi errati, inventati o non conformi allo schema predefinito. Invece di fidarsi ciecamente dell'output del modello, il sistema lo tratta come un "consiglio qualificato" che deve essere rigorosamente validato.

1° Strato di Difesa: Prompt Engineering

Il primo livello di controllo è preventivo e avviene prima ancora che l'LLM generi una risposta. La funzione `_build_prompt` costruisce un prompt estremamente specifico che agisce come un "contratto" per il modello:

```
"You are a librarian expert in book categorization. Your task is to
    analyze the provided book details and map its themes to a
   predefined set of categories.\n\n"
f"**Book Title:** {title}\n"
f"**Author(s):** {authors_str}\n"
f"**Description:** {truncated_description}\n"
f"**User-Generated Genres:** [{genres_str}]\n\n"
"Based on all the information above, map the book to one or more of
    the following predefined categories:\n"
f"[{predefined_genres_str}]\n\n"
"**Rules:**\n"
"1. Return a JSON object where keys are the matching predefined
   categories and values are always 1.\n"
"2. Prioritize the book's description and title over user-generated
    genres if they conflict.\n"
"3. If no specific category fits, but the genres or description
   suggest general fiction, use 'fiction' as a fallback.\n"
"4. If no category fits at all, return an empty JSON object {}.\n"
"Example for a sci-fi romance: {\"romance\": 1, \"fiction\": 1}\n"
"Your response must be only the JSON object, without any additional
    text or markdown."
```

2^0 Strato di Difesa: Validazione

Questo è il meccanismo di sicurezza più importante e agisce dopo aver ricevuto la risposta dall'LLM:

1. **Verifica della Struttura**: La funzione `map_genres` prima di tutto controlla se la risposta dell'LLM è un JSON valido e se ha la struttura "genere": 1, ... Se non lo è, l'intera risposta viene scartata.

2. **Iterazione e Validazione di Ciascun Genere**: Il sistema itera su ogni genere suggerito dall'LLM. Per ciascuno di essi, `_find_closest_predefined_genre` esegue due controlli in sequenza:

   (a) **Controllo di Corrispondenza Esatta**: Il sistema verifica se il genere restituito dall'LLM è identico ad una delle stringhe presenti nella lista PREDEFINED_GENRES_KEYS. Se lo è, il genere è considerato valido e viene approvato.

   (b) **Mappatura flessibile tramite Parole Chiave**: Se la corrispondenza esatta fallisce (es. l'LLM risponde: "Historical Biography" invece di "history, historical fiction, biography'), il sistema non scarta subito il genere. Tenta una mappatura "intelligente".

   - Analizza le parole contenute nel genere suggerito dall'LLM.
   - Controlla se queste parole corrispondono a delle parole chiave definite nel dizionario GENRE_KEYWORD_MAP.
   - Se trova una corrispondenza, mappa il suggerimento dell'LLM alla categoria predefinita corretta.

3. **Scarto Definitivo**: Se un genere suggerito dall'LLM fallisce entrambi i controlli, viene semplicemente ignorato e scartato. Non verrà mai incluso nel risultato finale.

3.6 Sistema di Raccomandazione

La progettazione e l'implementazione del sottosistema di raccomandazione rappresentano il nucleo funzionale di questa tesi. La scelta dell'approccio metodologico non è stata arbitraria, ma è una diretta conseguenza delle conclusioni emerse dalle fasi preliminari di studio e dei vincoli pratici del progetto.

Come evidenziato dall'analisi del dataset (Sezione 3.3.1), le risorse a disposizione presentano una duplice natura: da un lato, un vasto corpus di metadati; dall'altro, un'enorme mole di dati di interazione. Questa dualità, valutata alla luce dei modelli teorici discussi nella Sezione 2.2.2, ha reso evidente che affidarsi a una singola famiglia di algoritmi sarebbe stato limitante.

Per questo motivo, è stata adottata una strategia ibrida **memory-based**, progettando e implementando due motori di raccomandazione distinti ma complementari:

1. **Filtraggio Basato sul Contenuto**: Per sfruttare la ricchezza dei dati testuali (titoli, descrizioni, generi, tag) e fornire raccomandazioni basate sulla similarità intrinseca tra i libri.

2. **Filtraggio Collaborativo**: Per capitalizzare sulla vasta quantità di recensioni e scoprire preferenze emergenti, raccomandando libri apprezzati da utenti con gusti simili.

Contestualmente, è stata presa la decisione consapevole di escludere le metodologie **Model-Based** complesse. Data l'assenza di hardware specializzato e i limiti temporali, l'adozione di tali approcci avrebbe imposto una significativa riduzione del dataset, andando in diretta contraddizione con l'obiettivo di sfruttare appieno la ricchezza dei dati. Coerentemente con lo scopo di questa tesi, si è preferito puntare alla creazione di un'architettura performante e accessibile.

Pertanto, è stato scelto l'impiego di **Indici per la ricerca della *'similarità'*** al fine di produrre raccomandazioni. Questa scelta permette di pre-calcolare le rappresentazioni complesse dei dati in una fase offline, garantendo risposte in tempi brevi durante la fase di inferenza online.

3.6.1 Architettura del Sistema di Raccomandazione

L'architettura adottata si fonda su una separazione netta tra due ecosistemi:

1. **Ecosistema Offline**: Comprende due pipeline di elaborazione *una tantum* (o eseguite periodicamente) che trasformano i dati grezzi in indici di ricerca.
2. **Ecosistema Online**: Utilizza i modelli pre-calcolati per generare raccomandazioni in tempo reale, garantendo risposte rapide e a bassa latenza.

Ecosistema Offline: Costruzione degli Indici

La fase offline è responsabile della preparazione degli asset fondamentali su cui si basano le raccomandazioni. A causa della diversa natura dei dati — i metadati dei libri, relativamente statici, e i profili utente, intrinsecamente dinamici — sono state implementate due pipeline di costruzione distinte e specializzate.

Pipeline 1: Costruzione del Modello Content-Based (Indice Annoy) Questa pipeline trasforma il corpus di libri in uno spazio vettoriale interrogabile per similarità di contenuto.

- **Aggregazione delle Feature (`FeatureAggregator`):** Il primo passo consiste nel creare un "documento" testuale sintetico e ponderato per ogni libro. Il `FeatureAggregator` non si limita a concatenare titolo e descrizione, ma implementa una strategia di ponderazione delle feature. Vengono estratti i tag più rilevanti dai *popular shelves*, e la loro frequenza viene utilizzata per pesare la loro importanza. Titolo e generi principali vengono ripetuti più volte per conferirgli un peso maggiore nel documento finale, catturando così le sfumature tematiche di ogni opera.
- **Vettorizzazione e Indicizzazione (`ModelBuilder`):**
  - **Vettorizzazione TF-IDF:** I documenti testuali vengono trasformati in vettori numerici ad alta dimensionalità tramite l'algoritmo *Term Frequency-Inverse Document Frequency* (TF-IDF).

  - **Costruzione dell'Indice Annoy:** I vettori TF-IDF vengono indicizzati utilizzando `Annoy`, una libreria ottimizzata per la ricerca approssimata del vicino più prossimo (ANN) in scenari con dati statici. La metrica di *distanza angolare* viene utilizzata per misurare la similarità semantica.

- **Persistenza del Modello (`ModelPersister`):** L'intero modello `RecommenderModel`, che include il vettorizzatore TF-IDF, l'indice Annoy e le mappe di metadati, viene serializzato su disco, pronto per essere caricato rapidamente dalla fase online.

Pipeline 2: Costruzione dei Profili Utente e dell'Indice Collaborativo (Indice FAISS)
Questa pipeline, è progettata per creare una rappresentazione vettoriale dei gusti di ogni utente e indicizzarla per una ricerca efficiente di "utenti simili".

- **Calcolo dei Vettori dei Gusti (`TasteVectorCalculator`):** Per ogni utente nel database, il sistema recupera la sua cronologia di interazioni e calcola un *vettore dei gusti* (`taste_vector`). Questo vettore è una media dei vettori TF-IDF dei libri che l'utente ha valutato positivamente (rating ≥ 4).

- **Persistenza dei Profili (`UserProfileRepository`):** Ogni `taste_vector` calcolato viene salvato in una collection dedicata su MongoDB, associato all'ID dell'utente.

- **Costruzione dell'Indice FAISS (`UserProfileIndex`):** Tutti i profili utente vengono caricati e utilizzati per costruire un indice ANN con `FAISS`.

La scelta di utilizzare `FAISS` per i profili utente è una decisione architetturale strategica. A differenza dell'indice `Annoy` dei libri, che viene ricostruito interamente, `FAISS` è stato scelto per la sua capacità di supportare l'aggiunta incrementale di nuovi vettori a un indice esistente. Questa caratteristica è cruciale, poiché i profili utente sono dinamici: quando un nuovo utente si registra o un utente esistente valuta nuovi libri, il suo profilo può essere calcolato e aggiunto all'indice `FAISS` "a caldo", senza la necessità di ricalcolare l'intero indice da zero, garantendo così la scalabilità del sistema.

Ecosistema Online: Generazione delle Raccomandazioni

La fase online è orchestrata dalla classe `UserRecommenderFacade`, che agisce come punto di ingresso unificato (Pattern Facade) e seleziona la strategia di raccomandazione appropriata.

Strategia 1: Raccomandazione Content-Based Questo approccio genera suggerimenti basati esclusivamente sul profilo di gusti dell'utente target.

- **Profilazione Utente:** Viene calcolato il `taste_vector` dell'utente in tempo reale, basandosi sulla sua cronologia.
- **Query all'Indice Annoy:** Il `ContentBasedRecommender` utilizza il `taste_vector` per interrogare l'indice Annoy dei libri e recuperare una lista di candidati.
- **Affinamento tramite Re-Ranking:** La lista di candidati viene ulteriormente raffinata applicando una serie di strategie di re-ranking, come descritto di seguito.

Strategia 2: Raccomandazione Collaborative Filtering Questo approccio si basa sul principio che "gli utenti a cui sono piaciuti gli stessi libri potrebbero apprezzare anche altri libri simili".

- **Recupero o Creazione del Profilo Utente:** tramite `_get_or_create_user_profile`, recupera il `taste_vector` dell'utente. Se il profilo non esiste (nuovo utente), viene calcolato, salvato nel database e aggiunto dinamicamente all'indice FAISS in memoria.
- **Ricerca dei Vicini (Neighbors Search):** Il `CollaborativeFilteringRecommender` interroga l'indice FAISS degli utenti per trovare un insieme di "vicini", ovvero gli utenti con il `taste_vector` più simile a quello dell'utente target.
- **Aggregazione dei Candidati:** Il sistema analizza la cronologia dei libri preferiti dai "vicini" e aggrega i titoli che l'utente target non ha ancora letto, assegnando un punteggio a ogni candidato.

- **Re-Ranking:** La lista di candidati aggregati viene passata attraverso le stesse strategie di re-ranking utilizzate nel content-based, garantendo una personalizzazione finale coerente.

Affinamento tramite Re-Ranking (Pattern Strategy)

Entrambe le strategie di raccomandazione producono una lista grezza di candidati, che viene poi affinata per migliorarne la pertinenza. L'architettura utilizza il **Pattern Strategy**, definendo un'interfaccia `ReRanker` che permette di aggiungere o rimuovere nuove logiche di riordino in modo modulare. Le strategie implementate includono:

- **`PageCountReRanker`:** Assegna un bonus ai libri il cui numero di pagine rientra in un intervallo di circa ±20% rispetto alla media del numero di pagine dei libri preferiti dall'utente, evitando così di suggerire opere eccessivamente lunghe o troppo brevi rispetto alle sue abitudini.

- **`GenrePreferenceReRanker`:** Questa strategia implementa una personalizzazione più profonda, analizzando la cronologia delle valutazioni dell'utente per inferire le sue preferenze e avversioni specifiche per genere. Il processo si articola in due fasi: prima, durante la preparazione del contesto di re-ranking (metodo `_prepare_rerank_context`), il sistema analizza la cronologia dell'utente e costruisce due insiemi distinti: i generi *preferiti* (associati a libri con rating ≥ 4) e i generi *non graditi* (associati a libri con rating ≤ 2). Successivamente, per ogni libro candidato nella lista di raccomandazione, il re-ranker applica un bonus al suo punteggio di similarità per ogni genere in comune con i preferiti e un malus per ogni genere in comune con i non graditi. Questo meccanismo permette di promuovere attivamente i libri che rispecchiano i gusti specifici dell'utente e di penalizzare quelli che appartengono a categorie che ha dimostrato di non apprezzare, sfavorendo quindi libri che possiedono generi o sub-generi non graditi.

3.7 WebApp

La webapp è stata sviluppata per fornire le funzionalità principali a disposizione di un utente che accede alla piattaforma: autenticazione tramite username e password (login e registrazione), aggiunta, modifica e rimozione di libri valutati, ricerca di libri per titolo o autore, e richiesta di raccomandazioni basate sulle valutazioni effettuate. Queste funzionalità sono state realizzate principalmente per supportare le attività di testing del sistema di raccomandazione. Poiché questa componente non rappresenta un contributo significativo alla tesi, vengono riportati esclusivamente alcuni screenshot (3.4, 3.5, 3.6) esemplificativi dell'implementazione completa.

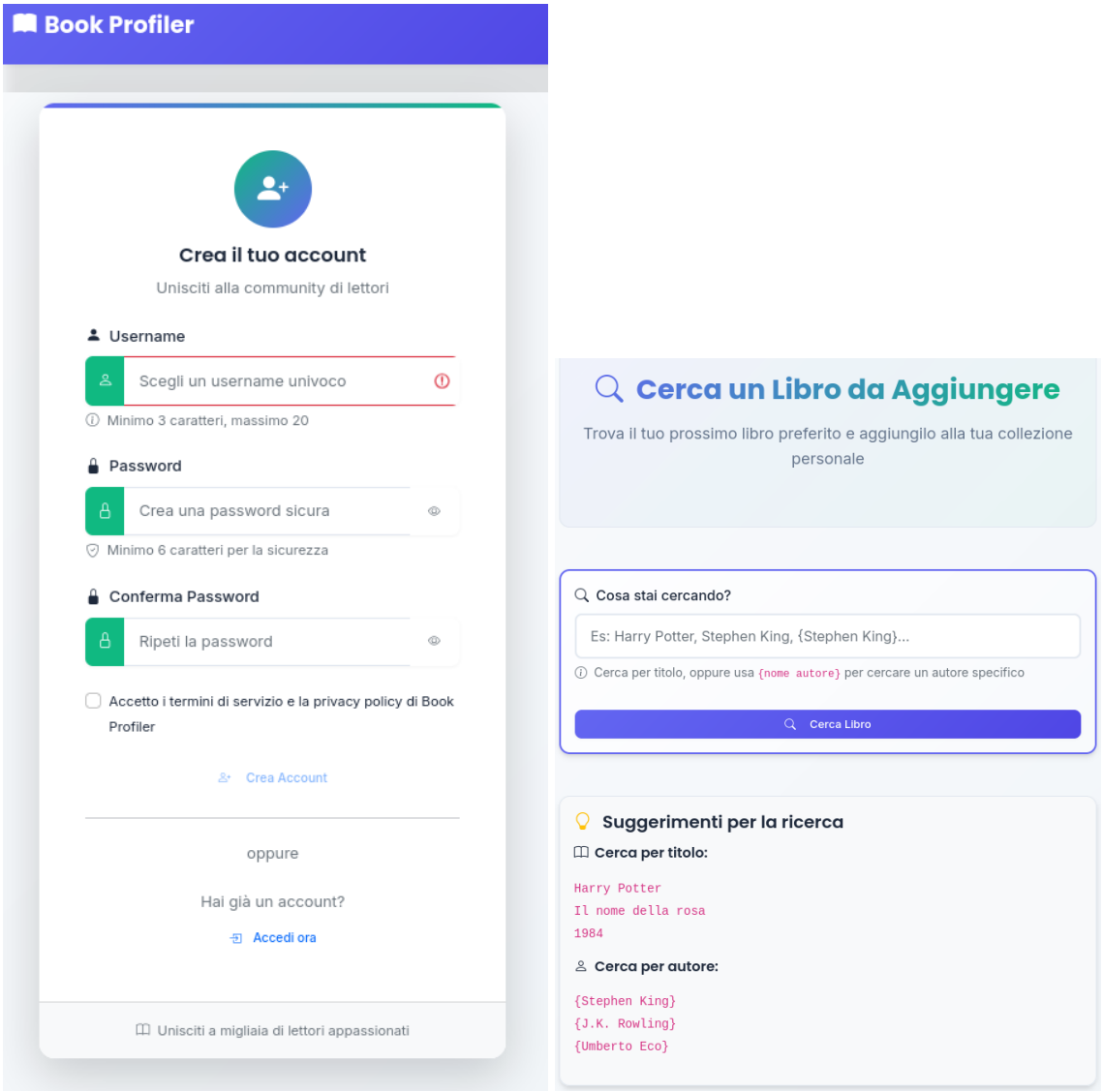


Figura 3.4: Pagine di Sign-Up e Ricerca

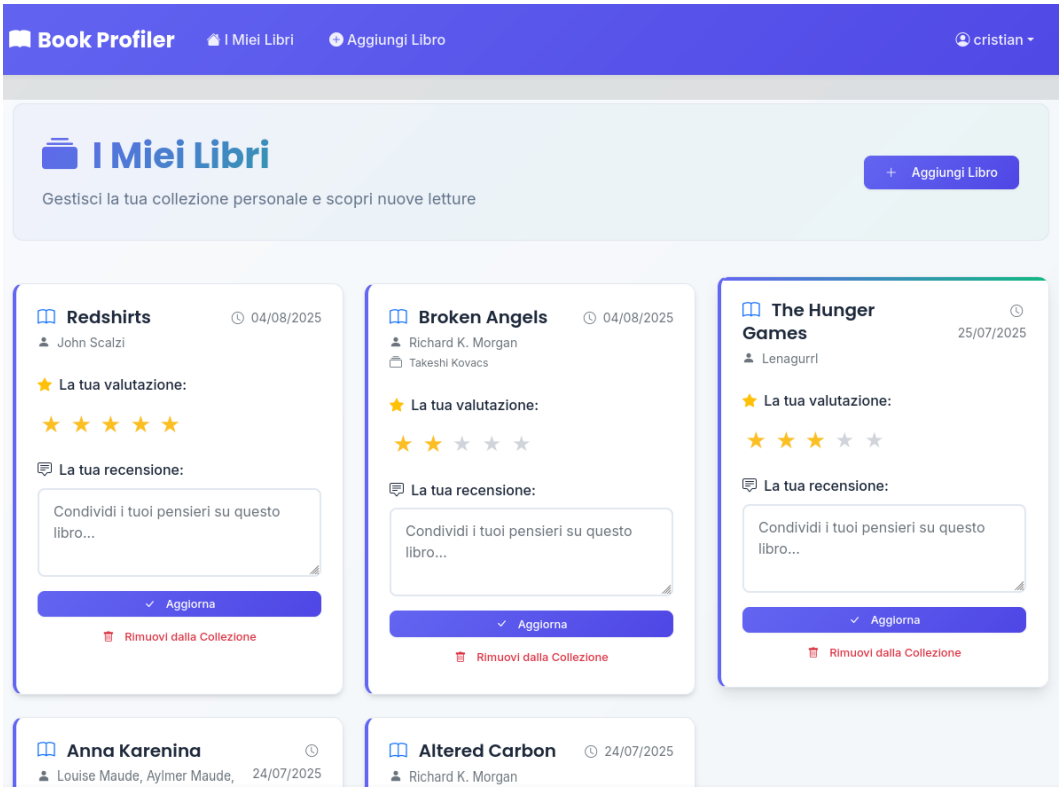


Figura 3.5: Catalogo personale

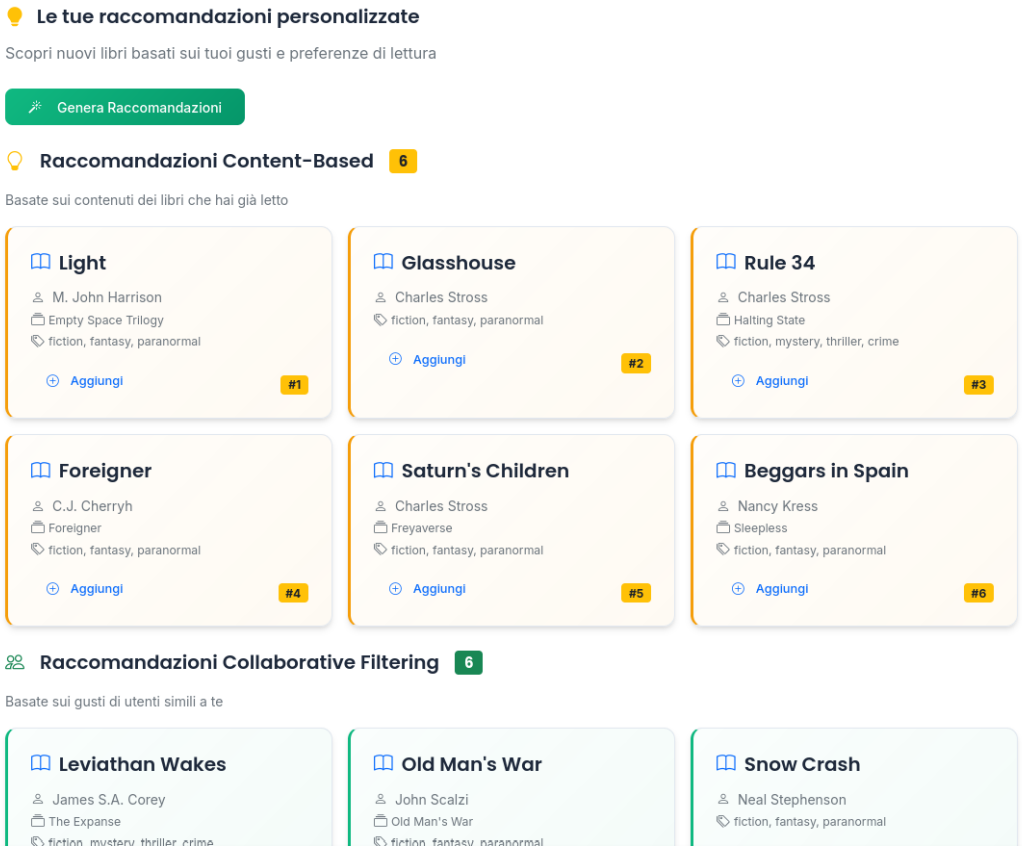


Figura 3.6: Visualizzazione delle Raccomandazioni

CAPITOLO 4

Valutazione dell'Applicazione

4.1 Sistema di Data Augmentation

Questa sezione analizza le performance del sistema di Data Augmentation nell'efficacia qualitativa del processo di mappatura dei generi.

4.1.1 Metodologia di Test e Dataset

Per validare il sistema in uno scenario realistico, è stato costruito un dataset di test basato sulle "New Releases" pubblicate sul portale Goodreads mensilmente[1]. È stato scelto un campione rappresentativo che copre il secondo semestre del 2024 (da Luglio a Dicembre).

Il dataset finale comprende per le 11 categorie (non predefinite) * 5 libri * 6 mesi = 330 libri unici, rappresentativi delle nuove uscite in un ampio spettro di generi similari al dataset di partenza: *Fiction*, *Historical Fiction*, *History & Biography*, *Memoir & Autobiography*, *Mystery & Thriller*, *Romance*, *Fantasy*, *Science-Fiction*, *Non-Fiction*, *Young Adult*, *Graphic Novels & Comics*.

[1]New Releases `goodreads.com/new_releases/2024/7`

Fornire una Tassonomia Controllata

Il problema centrale che questo sistema è stato progettato per risolvere è l'inconsistenza dei generi nel mondo reale. Le fonti dati **recuperate esclusivamente** da Goodreads con l'apposito Provider non utilizzano una tassonomia rigida, ma una folksonomia, ovvero un sistema di etichettatura libero basato sui tag inseriti dagli utenti.

L'analisi del campione di 330 libri ha confermato la magnitudine di questa sfida:

- Il sistema ha estratto un totale di **263 etichette** di genere **uniche** e non normalizzate.
- Queste etichette variavano da generi ampi ("Fiction") a sotto-generi estremamente specifici ("Fantasy > Urban Fantasy", "History > World War II") fino a tag tematici o soggettivi ("book-club", "read-for-school").

Senza un meccanismo di normalizzazione, questi dati sarebbero stati inutilizzabili per un sistema di raccomandazione. L'obiettivo del sistema era quindi quello di mappare questa complessità nelle 10 macro-categorie predefinite dal sistema.

Analisi dei Risultati

Il grafico in Figura 4.1 mostra la distribuzione dei 330 libri di test all'interno delle 10 categorie predefinite, dopo essere stati processati dal sistema. L'analisi del grafico rivela diversi punti chiave sull'efficacia del *Genre Mapper*.

- **Distribuzione Coerente dei Generi**: La categoria "fiction" emerge come la più popolosa, con oltre 160 libri. Questo risultato è atteso e corretto, poiché "fiction" agisce sia come genere a sé stante sia come "sub-genere" per altri generi letterari. Seguono le altre categorie principali come "history, historical fiction, biography" che accorpa ben 3 categorie 'Historical Fiction, History & Biography, Memoir & Autobiography'.

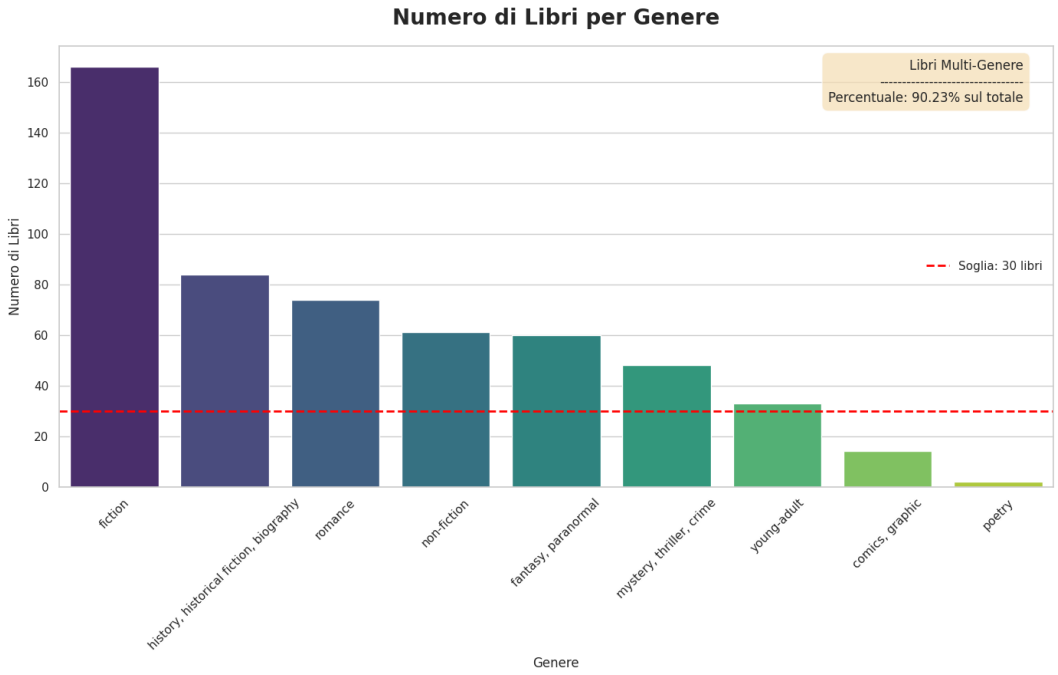


Figura 4.1: Distribuzione dei generi

- **Categorie Sottorappresentate**: Ad un'attenta analisi del grafico, si nota che la categoria *"comics, graphics"* contiene un numero di volumi inferiore a 20, significativamente al di sotto dei 30 libri che costituivano il campione di test per ciascuna categoria, tra cui 'Graphic Novels & Comics". Questa discrepanza non rappresenta un vero fallimento del mapper, ma evidenzia una **limitazione intrinseca e prevedibile** di un'analisi basata esclusivamente su metadati testuali.

  – L'LLM, operando su titolo, descrizione e generi testuali, è eccezionalmente abile a cogliere temi, ma è "cieco" al formato visuale di un'opera. Non può "saperlo" se ciò non è esplicitamente e inequivocabilmente dichiarato nei metadati testuali.

  – Pertanto vengono invece categorizzati in base al loro contenuto narrativo. La sottorappresentazione della categoria *comics, graphic* è una conseguenza della natura testuale dell'input, che rafforza la capacità del sistema anche quando le indicazioni sul formato sono deboli.

  La categoria *"poetry"* invece risulta essere non prevista, evidenziando un minimo errore di etichettatura.

4.1.2 Confronto Quantitativo con altri Providers

Per valutare in modo oggettivo il grado di coerenza tra i generi estratti precedentemente (DB1) e quelli aggregati dai provider GoogleBooks e OpenLibrary (DB2), è stato calcolato l'Indice di Jaccard per i 158 (dei 330) libri presenti in entrambi i database. Questa metrica, che misura la similarità tra due insiemi di generi (A e B) è definita come:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Un valore di 1.0 indica un accordo perfetto (gli insiemi di generi sono identici), mentre un valore di 0.0 indica un disaccordo totale (gli insiemi non hanno generi in comune).

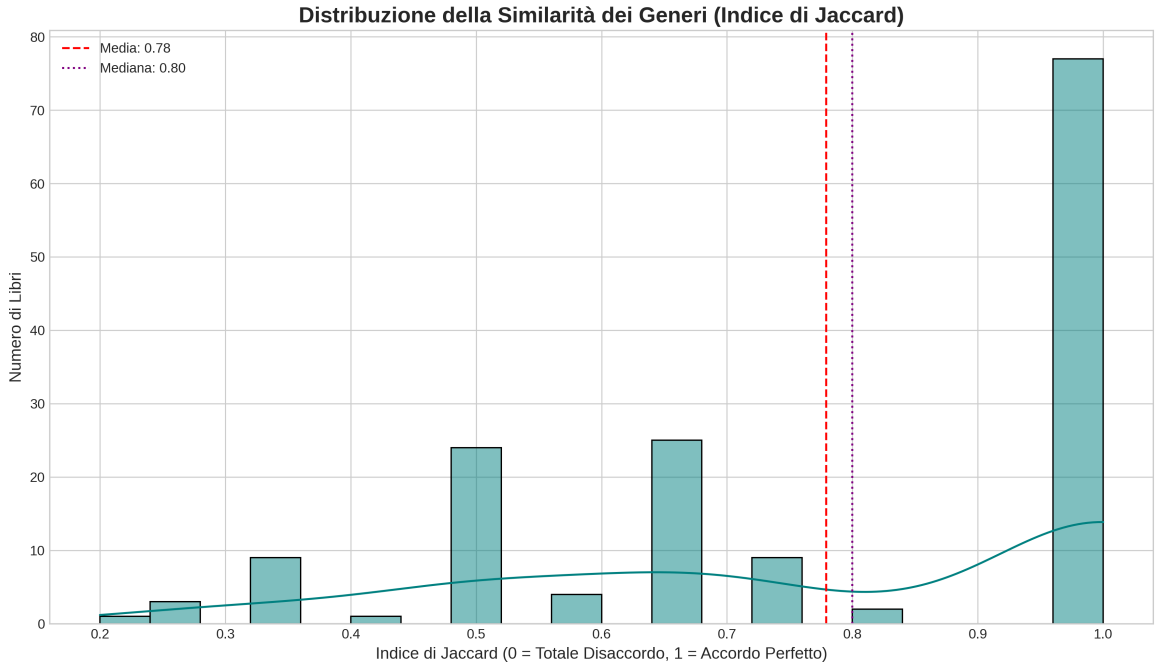


Figura 4.2: Distribuzione dell'Indice di Jaccard per tutti i libri in comune.

Il risultato di questa analisi è visualizzato nell'istogramma in Figura 4.2, che offre una visione chiara e quantitativa delle performance del sistema di mappatura. L'analisi del grafico rivela diverse intuizioni fondamentali:

- **Dominanza dell'Accordo Perfetto:** La stragrande maggioranza dei libri analizzati (quasi 80 su 158), l'insieme di generi mappati dal sistema partendo dai dati di Goodreads è esattamente identico a quello ottenuto partendo dai dati degli altri provider a dimostrazione che la logica di mappatura basata su

LLM è determinista e coerente, capace di normalizzare informazioni diverse e disordinate verso la stessa, identica tassonomia controllata.

- **Elevata Similarità Complessiva**: Includendo anche i match perfetti, la media *(0.78)* e la mediana *(0.80)* dei punteggi di similarità sono estremamente alte. Questo dato conferma che il sistema non solo è coerente, ma produce risultati altamente simili anche quando non sono identici.

- **Natura delle Divergenze Minoritarie**: I picchi secondari, visibili a valori di Jaccard di circa 0.5 e 0.67, rappresentano i casi di "match parziale". Queste discrepanze non sono tipicamente indice di un errore, ma di una differenza di granularità tra le fonti. Ad esempio, un libro potrebbe essere etichettato come *"fiction"*, *"romance"* da un sistema e *"fiction"*, *"romance"*, *"contemporary"* dall'altro. Questo si traduce in un indice di Jaccard di $2/3$ (≈ 0.67), che il grafico mostra come un picco significativo.

- **Insignificanza dei Disaccordi Forti**: La coda sinistra del grafico (Jaccard < 0.4) è quasi trascurabile. Ciò dimostra in modo inequivocabile che i casi in cui le fonti forniscono informazioni sui generi che sono in totale o quasi totale conflitto sono un'anomalia rara.

Tuttavia, la scoperta più significativa di questa analisi emerge dal confronto della qualità dei dati in ingresso. Il provider primario, GoodreadsRustScraper (DB1), ha fornito un insieme di 263 generi unici su 330 libri (237 sui 158 libri in comune), offrendo un'enorme ricchezza di dettagli e sfumature. Al contrario, l'insieme dei provider secondari (DB2) ha prodotto un vocabolario molto più limitato, con solo 59 generi unici su 158 dei 330 libri reperibili. Nonostante questa drastica disparità nella ricchezza dei dati di partenza, il sistema è stato in grado di produrre una classificazione finale quasi identica per entrambi i set di dati. Questo risultato è fondamentale e dimostra che: l'LLM non sta semplicemente eseguendo una mappatura basata su parole chiave, ma sta sfruttando la sua comprensione semantica del titolo e della descrizione del libro per inferire le categorie corrette, compensando attivamente la povertà e la mancanza di specificità dei dati di input.

4.2 Sistema di Raccomandazione

La validazione dell'efficacia del sistema di raccomandazione è stata condotta attraverso una meticolosa metodologia di valutazione offline, orchestrata dallo script `offline_evaluator.py`. Questa sezione descrive in dettaglio tale metodologia e presenta i risultati ottenuti attraverso un processo di ottimizzazione iterativo.

4.2.1 Metodologia di Valutazione Offline

L'approccio scelto è una strategia di *hold-out*, uno standard del settore per la valutazione di sistemi di raccomandazione. Il processo è stato applicato a un campione casuale di circa 1000 utenti (l'aumento non influenzava i risultati), selezionati secondo criteri specifici per garantire la robustezza statistica dei risultati. La procedura per ogni utente è la seguente:

1. **Selezione dell'Utente:** vengono considerati solo utenti con un minimo di 20 recensioni (`min_reviews=20`), per assicurare che il loro profilo di gusti sia sufficientemente ricco.

2. **Verifica per lo Split:** l'utente deve aver valutato positivamente (`rating` $\geq$ 4) almeno 10 libri (`min_high_ratings_for_split=10`). Questo garantisce che ci sia un numero sufficiente di item "amati" per creare un insieme di test significativo.

3. **Creazione del Set di Hold-Out:** Dalla lista dei libri valutati positivamente, ne vengono "nascosti" 5 in modo casuale (`holdout_size=5`). Questi 5 libri costituiscono la "verità oggettiva" (*ground truth*) che il sistema dovrà tentare di raccomandare.

4. **Generazione delle Raccomandazioni:** il sistema genera una lista di `k` raccomandazioni (nei test, `k=10` o `k=20`) utilizzando la restante cronologia dell'utente come input.

5. **Calcolo delle Metriche:** le raccomandazioni vengono confrontate con il set di hold-out per calcolare le metriche di performance.

Le metriche chiave utilizzate sono la **Precision@k**, il **Recall@k** e la **Copertura del Catalogo**.

Nota sui Limiti della Valutazione Offline È fondamentale sottolineare i limiti intrinseci di questa metodologia. Essa si basa sull'assunto che solo i libri "nascosti" siano raccomandazioni corrette, ignorando migliaia di altri titoli nel catalogo che potrebbero risultare altrettanto, se non più, graditi all'utente. La raccomandazione libraria non è un problema con una singola soluzione esatta come le previsioni meteo, ma dipende da fattori soggettivi. Per mitigare questa limitazione, la valutazione quantitativa è stata affiancata da un'analisi qualitativa tramite un questionario somministrato a beta-tester, volto a misurare la capacità del sistema di generare interesse nella lettura di nuovi titoli. Maggiori informazioni nella Sezione 4.2.2.

Fase 1: Ottimizzazione delle Rappresentazioni Vettoriali

Il punto di partenza è stato un modello baseline con una dimensione dello spazio vettoriale limitata a 500. Come mostrato nella Tabella 4.2, i risultati iniziali hanno rivelato ampi margini di miglioramento per la strategia **Collaborative-Filtering**.

La prima ottimizzazione ha riguardato l'aumento della dimensione dei vettori a **4500**. Successivamente, il `TasteVectorCalculator` è stato modificato per basarsi esclusivamente sui libri valutati positivamente (`rating` ≥ 4). Questi interventi hanno consolidato le fondamenta del sistema, producendo un impatto notevole sulle performance del sistema **Content-Based**, la cui `Precision@10` è aumentata complessivamente del **500%** (da `0.0010` a `0.0061`).

Fase 2: Valutazione delle Strategie di Re-Ranking

Una volta stabilita una solida base vettoriale, sono stati isolati e valutati i due moduli di re-ranking. I risultati per la strategia collaborativa sono riassunti nella Tabella 4.1.

Dall'analisi emerge che il `PageCountReRanker` apporta un **beneficio** misurabile, aumentando l'accuratezza senza sacrificare la diversità.

Tabella 4.1: Confronto delle performance (Collaborative Filtering, k=10) con diverse strategie di Re-Ranking.

Configurazione	Avg Precision@k	Avg Recall@k	Catalog Coverage
Senza Re-Ranker	0.0225	0.0451	0.62%
Solo PageCountReRanker	**0.0241**	**0.0483**	**0.59%**
Solo GenrePreferenceReRanker	0.0213	0.0426	0.33%

Al contrario, il **`GenrePreferenceReRanker` risulta dannoso**, riducendo sia la precisione sia la copertura del catalogo, agendo come un filtro eccessivamente rigido che soffoca la scoperta di nuovi interessi (*serendipity*).

Sintesi dei Risultati e Configurazione Ottimale

La Tabella 4.2 riassume l'evoluzione delle performance durante l'intero processo di ottimizzazione, partendo dal modello iniziale fino alla configurazione finale, che rappresenta lo stato dell'arte per l'architettura TF-IDF sviluppata in questa tesi. La configurazione ottimale adotta uno spazio vettoriale a 4500 dimensioni, un calcolo del `taste_vector` basato solo su valutazioni positive e l'uso del solo `PageCountReRanker`.

Tabella 4.2: Evoluzione delle metriche di performance durante il processo di ottimizzazione.

Esperimento	k	Avg Precision@k		Avg Recall@k		Coverage
		CB	CF	CB	CF	
1. Baseline (Vettori a 500)	10	0.0010	0.0219	0.0020	0.0438	0.26%
2. Espansione Vettoriale (4500)	10	0.0040	0.0204	0.0080	0.0408	0.41%
3. `TasteVector` Solo Positivi	10	0.0061	0.0182	0.0122	0.0363	0.33%
4. Configurazione Finale + k=20	**20**	**0.0037**	**0.0186**	**0.0147**	**0.0744**	**0.85%**

Questo ciclo di esperimenti ha permesso di ottimizzare ogni componente del sistema, raggiungendo il punto di *massima* performance per l'architettura basata su

TF-IDF e filtraggio collaborativo *memory-based*. I risultati ottenuti costituiscono una baseline solida.

Tuttavia, il *plateau* di performance raggiunto indica chiaramente che la media di vettori TF-IDF non è un meccanismo sufficientemente sofisticato per catturare la complessità del gusto di un utente né la similarità tra i libri.

Fase 3: Valutazione Avanzata con Analisi dell'Allineamento di Genere

Per completare la valutazione del sistema, è stata implementata una metodologia di valutazione avanzata attraverso il modulo `enhanced_offline_evaluator.py`, che estende la valutazione tradizionale con due innovazioni fondamentali: l'**analisi dell'allineamento di genere** e il **confronto tra strategie di suddivisione temporale e casuale** dei dati.

Metodologia dell'Analisi dell'Allineamento di Genere L'analisi dell'allineamento di genere, implementata tramite la classe `GenreAlignmentAnalyzer`, valuta quanto le raccomandazioni generate siano coerenti con le preferenze di genere storiche dell'utente. Questa analisi introduce quattro metriche complementari:

- **Genre Overlap**: Misurata tramite coefficiente di Jaccard tra i generi dei libri raccomandati e quelli preferiti storicamente dall'utente.
- **Genre Diversity**: Numero totale di generi distinti presenti nelle raccomandazioni, indicatore della varietà offerta dal sistema.
- **Popularity Bias**: Frazione delle raccomandazioni che coincidono con i libri più popolari del catalogo, misura del rischio di omologazione.
- **Genre Match Score**: Percentuale di raccomandazioni che appartengono ai generi preferiti dell'utente.

Confronto tra Suddivisione Temporale e Casuale La metodologia tradizionale utilizza una suddivisione *casuale* dei dati storici dell'utente per creare il set di hold-out. Tuttavia, questa approccio non riflette la realtà d'uso del sistema, dove

le raccomandazioni devono predire le future preferenze basandosi sulla cronologia passata.

Per questo motivo, è stata implementata una **suddivisione temporale** che:

1. Ordina cronologicamente tutte le interazioni dell'utente utilizzando i campi `date_added`, `date_updated` o `review_date`.
2. Utilizza l'80% delle interazioni più recenti per l'addestramento.
3. Riserva il 20% delle interazioni più recenti come set di test.

Questa metodologia fornisce una valutazione più realistica della capacità predittiva del sistema nel contesto di utilizzo reale.

Risultati della Valutazione Avanzata I risultati ottenuti attraverso la valutazione avanzata su un campione di circa 190 utenti sono riportati nella Tabella 4.3. I dati confermano le aspettative teoriche: la **suddivisione temporale produce performance superiori** rispetto alla suddivisione casuale, poiché riflette più fedelmente lo scenario di utilizzo reale del sistema.

Tabella 4.3: Confronto tra suddivisione casuale e temporale con analisi dell'allineamento di genere.

Metrica	Content-Based		Collaborative Filtering	
	Casuale	Temporale	Casuale	Temporale
Avg Precision@10	0.003	0.005	0.013	0.022
Avg Recall@10	0.009	0.018	0.043	0.072
Genre Overlap	0.424	0.413	0.454	0.441
Genre Diversity	221.9	219.2	231.0	224.0
Popularity Bias	0.059	0.059	0.053	0.054
Genre Match Score	0.998	0.999	1.000	1.000

Interpretazione dei Risultati dell'Analisi di Genere L'analisi dell'allineamento di genere rivela caratteristiche interessanti del sistema:

- **Genre Overlap** intorno a 0.4-0.45 indica che circa il 40-45% dei generi raccomandati coincidono con le preferenze storiche dell'utente, suggerendo un buon equilibrio tra coerenza e scoperta.
- **Genre Diversity** elevata (>200 generi distinti) dimostra che il sistema non soffre di eccessiva specializzazione e mantiene una buona varietà nelle raccomandazioni.
- **Popularity Bias** contenuto (5-6%) conferma che il sistema non è eccessivamente influenzato dalla popolarità globale dei titoli.
- **Genre Match Score** prossimo al 100% indica che quasi tutte le raccomandazioni appartengono a generi già apprezzati dall'utente, suggerendo una forte capacità di personalizzazione.

Il **Collaborative Filtering** mostra performance superiori in tutte le metriche tradizionali rispetto al **Content-Based**, confermando i risultati delle fasi precedenti. La suddivisione temporale, come atteso, produce performance superiori in entrambi gli approcci, validando l'importanza di metodologie di valutazione più realistiche.

Questa analisi avanzata completa il quadro valutativo del sistema, fornendo insights sia quantitativi che qualitativi sulla capacità del sistema di bilanciare personalizzazione, diversità e scoperta di nuovi contenuti.

4.2.2 Valutazione Qualitativa tramite Beta-Testing

Per completare la valutazione del sistema di raccomandazione, è stata condotta una **valutazione qualitativa** attraverso un questionario[2] distribuito a beta-tester reali. Questa fase rappresenta un complemento fondamentale alla valutazione quantitativa offline, in quanto misura la capacità del sistema di generare interesse verso nuovi titoli in scenari d'uso realistici.

Metodologia del Beta-Testing La valutazione qualitativa è stata strutturata come segue:

1. **Tempo**: dall'11 Agosto al 23 Agosto.
2. **Campione**: 27 utenti beta-tester, amanti dei libri reclutati tramite gruppi Telegram dedicati ai libri con diversi profili di lettura hanno partecipato volontariamente al questionario.
3. **Procedura**: Ogni partecipante, dopo aver inserito tra le 10 e 20 recensioni nella propria libreria ha ricevuto 10 raccomandazioni personalizzate (5 generate dal sistema **Content-Based** e 5 dal **Collaborative Filtering**). Successivamente, si sarebbe informato su ogni libro raccomandato dal sistema.
4. **Metriche qualitative**: Per ogni libro consigliato, gli utenti hanno fornito:
    - **Familiarità**: Scala categorica ("L'ho già letto", "Ne ho sentito parlare ma non l'ho letto", "Non lo conoscevo").
    - **Interesse**: Scala Likert 1-5 sulla propensione a leggere il libro.
    - **Feedback aperto**: Commenti testuali liberi su ogni raccomandazione.
5. **Valutazione generale**: Soddisfazione complessiva, percezione della diversità delle raccomandazioni, intenzione di utilizzo futuro della piattaforma, commento facoltativo sommario.

[2]Google Form https://forms.gle/9pqhb2wV4kATPszN7

Risultati

- **Response Rate**: Su una somma totale di 1076 utenti, solo il 2.5% ha partecipato al sondaggio.
- **Distribuzione di Familiarità**: Il 62.1% delle raccomandazioni riguarda libri di cui gli utenti hanno sentito parlare ma non hanno letto, il 33.1% libri completamente sconosciuti e solo il 4.8% libri già letti. Questa distribuzione è ottimale per un sistema di raccomandazione, in quanto indica una buona capacità di *discovery* (33.1% di contenuti nuovi) bilanciata con raccomandazioni di titoli noti ma non ancora esplorati.
- **Distribuzione di Interesse**: La distribuzione mostra una forte concentrazione sui valori alti della scala (4 e 5), con una media complessiva di 3.7. Questo risultato conferma quantitativamente la capacità del sistema di generare raccomandazioni che suscitano effettivo interesse negli utenti.

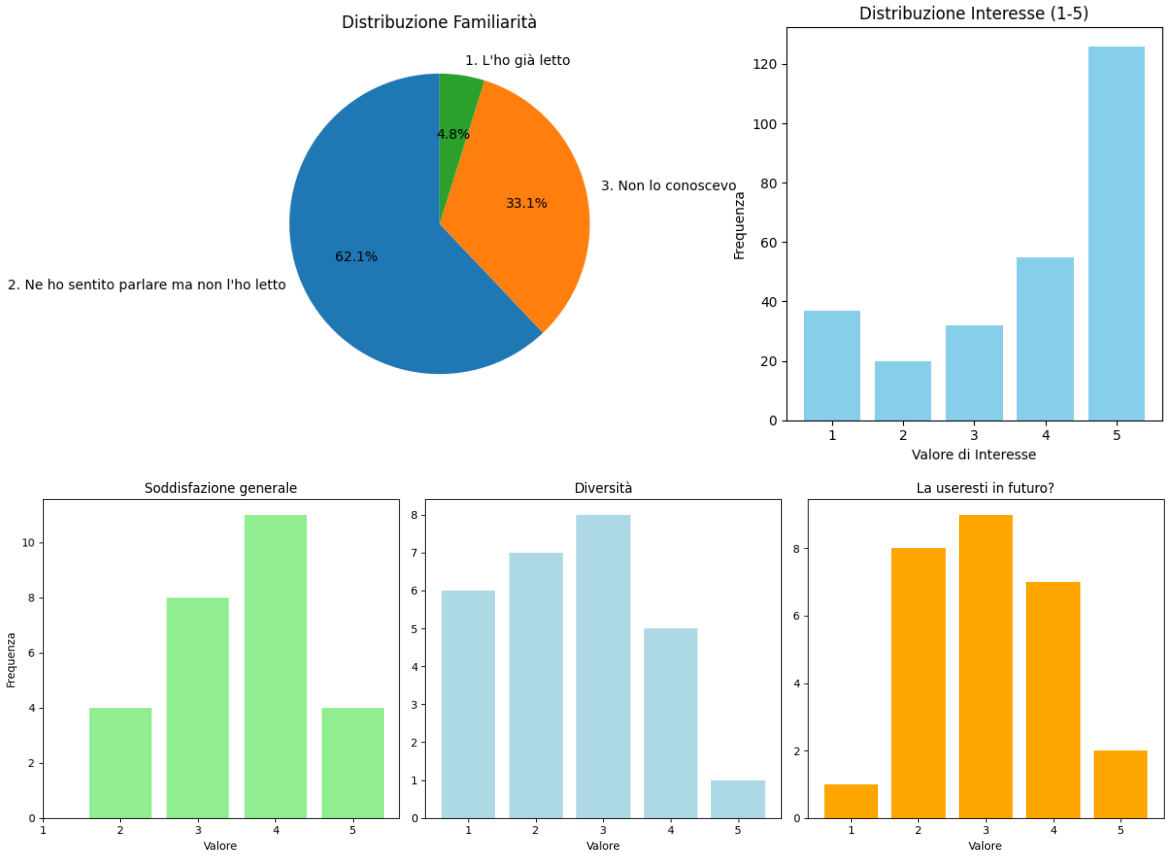


Figura 4.3: Grafici delle risposte ai questionari.

Analisi Qualitativa dei Feedback Testuali L'analisi dei commenti aperti fornisce insights preziosi sui punti di forza e le criticità del sistema:

Punti di Forza Identificati:

- **Identificazione accurata delle macro-aree di interesse**: Gli utenti riconoscono che il sistema coglie correttamente i generi letterari di loro preferenza.
- **Capacità di scoperta**: Diversi utenti esprimono entusiasmo per raccomandazioni di titoli sconosciuti che risultano per loro attraenti.
- **Bilanciamento tra familiarità e novità**: Il sistema riesce a proporre sia classici noti sia opere meno popolari.

Criticità Ricorrenti:

- **Gestione inadeguata delle serie**: Il problema più frequentemente segnalato è la raccomandazione di sequel non ancora iniziate dall'utente.
- **Presenza di duplicati**: Multiple occorrenze dello stesso titolo nella stessa lista di raccomandazioni minano la fiducia nel sistema. Lo stesso libro è stato suggerito sia dal Content-Based sia dal Collaborative Filtering.
- **Mancanza di filtri linguistici**: Per utenti con preferenze linguistiche specifiche (es. Spagnolo), il sistema non applica filtri adeguati.

Confronto tra Content-Based e Collaborative Filtering Dai feedback emerge una percezione differenziata delle due strategie:

- Il **Collaborative Filtering** viene spesso percepito come più accurato nel cogliere sfumature sottili dei gusti.
- Il **Content-Based** viene apprezzato per la coerenza tematica, ma criticato per la rigidità (poca *diversity)*.

Implicazioni per lo Sviluppo Futuro La valutazione qualitativa conferma i risultati della valutazione quantitativa e fornisce direzioni chiare per il miglioramento:

1. **Priorità assoluta**: Implementazione di un sistema intelligente di gestione delle serie che raccomandi sempre il primo volume non letto.

2. **Personalizzazione avanzata**: Introduzione di filtri configurabili dall'utente (lingua, formato) per rispettare preferenze specifiche.

La valutazione qualitativa dimostra che, nonostante le criticità tecniche identificate, il sistema possiede un solido potenziale nell'identificare e soddisfare le preferenze degli utenti, confermando la validità dell'approccio architetturale adottato.

CAPITOLO 5

Conclusioni

Portare a termine questo progetto rappresenta per me una profonda soddisfazione, in quanto coincide con il traguardo conclusivo del mio percorso di studi e mi ha permesso di coniugare il rigore accademico con una passione personale per la lettura e l'informatica.

Lo sviluppo del sistema di raccomandazione mi ha dato l'opportunità di approfondire tematiche come l'elaborazione dei dati e i modelli di raccomandazione odierni, affrontando al tempo stesso sfide legate alle prestazioni e all'esperienza d'uso. Il risultato ottenuto è un prototipo funzionante, facilmente replicabile su altre macchine attraverso i file di configurazione, capace di offrire raccomandazioni personalizzate e di aggiungere nuovi libri all'archivio con una qualità apprezzabile. I risultati ottenuti confermano la validità delle scelte progettuali adottate e rappresenta il mio contributo alla progettazione di sistemi di raccomandazione su dataset aggiornabili.

Il progetto è disponibile alla pagina GitHub:
github.com/panuozzo77/internship-book-recommending-system

5.1 Sviluppi Futuri

Il progetto è stato sapientemente limitato date le tempistiche e le capacità hardware a disposizione ma sarebbe possibile migliorarlo ulteriormente:

- **Suite di Test**: implementare test di unità, integrazione e sistema per garantire robustezza e affidabilità del codice.
- **Architettura a Microservizi**: evolvere il sistema da un'architettura monolitica a una basata su microservizi, migliorando scalabilità e manutenibilità.
- **Funzionalità WebApp**: introdurre ricerca avanzata con filtri, pagine dedicate per ciascun libro, gestione completa degli account e un pannello di amministrazione.
- **Filtraggio dei Sequel**: evitare di raccomandare sequel di saghe non ancora lette dall'utente.
- **Sperimentazione di nuovi modelli**: valutare algoritmi più avanzati, includendo nuovi approcci ibridi e tecniche di deep learning.
- **Aggiornamenti automatici della libreria**: invece di curare manualmente la lista degli inserimenti, attraverso eventi come RSS feed, aggiornare di conseguenza il catalogo librario.

In conclusione, il progetto conferma l'efficacia dei piccoli LLM nei compiti di etichettatura e l'utilità di keyword relative a genere, formato e tag per fornire nuovi libri interessanti agli utenti.

Bibliografia

[1] Zanichelli, Dizionari Più, Online ed. Zanichelli, 2015, lettore forte, chi legge annualmente molti più libri rispetto alla media. [Online]. Available: https://dizionaripiu.zanichelli.it/cultura-e-attualita/le-parole-del-giorno/parola-del-giorno/lettore/ (Citato a pagina 1)

[2] K. Smith-Yoshimura, Il passaggio ai metadati di nuova generazione. Dublin, OH: OCLC Research, 2021, tradotto da Smartling, Inc., Maria Cristina Mencato e Sabrina Masoli. Edizione italiana ottobre 2021. Edizione originale settembre 2020. [Online]. Available: https://doi.org/10.25333/k23b-hd86 (Citato a pagina 2)

[3] Y. Tang, R. Zhang, J. Guo, M. de Rijke, S. Liu, S. Wang, D. Yin, and X. Cheng, "Generative retrieval for book search," arXiv preprint arXiv:2305.13189, 2025, preprint available at arXiv:2305.13189. [Online]. Available: https://arxiv.org/pdf/2501.11034 (Citato a pagina 5)

[4] J. Moreci, Shut Up and Write the Book: A Step-by-Step Guide to Crafting Your Novel from Plan to Print. Independently published, 2023, capitolo 22. (Citato a pagina 6)

[5] S. Celia, M. Guarascio, M. Minici, F. Palopoli, and E. Ritacco, "Tecniche innovative di machine learning per la fruizione di contenuti digitali,"

Consiglio Nazionale delle Ricerche, Istituto di Calcolo e Reti ad Alte Prestazioni (ICAR), Via P. Bucci 8-9C, 87036 Rende, Italy, Technical Report RT-ICAR-CS-21-06, June 2021, iCAR-CNR, sedi di Cosenza, Napoli e Palermo. [Online]. Available: https://intranet.icar.cnr.it/wp-content/uploads/2021/07/RT-ICAR-CS-21-06.pdf (Citato a pagina 8)

[6] F. Ricci, L. Rokach, and B. Shapira, Recommender Systems Handbook, 3rd ed. Springer, 2015. (Citato alle pagine 11 e 16)

[7] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "Grouplens: an open architecture for collaborative filtering of netnews," in Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work, ser. CSCW '94. New York, NY, USA: Association for Computing Machinery, 1994, p. 175–186. [Online]. Available: https://doi.org/10.1145/192844.192905 (Citato a pagina 13)

[8] A. D. Biasio, A. Montagna, F. Aiolli, and N. Navarin, "A systematic review of value-aware recommender systems," Expert Systems with Applications, vol. 226, p. 120131, 2023. [Online]. Available: https://doi.org/10.1016/j.eswa.2023.120131 (Citato a pagina 14)

[9] M. Wan, R. Misra, N. Nakashole, and J. J. McAuley, "Fine-grained spoiler detection from large-scale review corpora," Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pp. 2605–2610, 2019. [Online]. Available: https://doi.org/10.18653/v1/p19-1248 (Citato a pagina 20)

[10] M. Wan and J. J. McAuley, "Item recommendation on monotonic behavior chains," Proceedings of the 12th ACM Conference on Recommender Systems, RecSys 2018, Vancouver, BC, Canada, October 2-7, 2018, pp. 86–94, 2018. [Online]. Available: https://doi.org/10.1145/3240323.3240369 (Citato a pagina 20)

[11] NVIDIA NIM, "gemma-3-27b-it model by google," https://build.nvidia.com/google/gemma-3-27b-it/modelcard, 2025, model card via NVIDIA NIM. (Citato a pagina 39)

[12] Gemma Team, "Gemma 3 technical report," arXiv preprint arXiv:2503.19786, Tech. Rep., 2025. [Online]. Available: https://arxiv.org/abs/2503.19786 (Citato a pagina 39)

[13] M. Gupta, "Google gemma3: The best non-reasoning llm," Medium (Data Science in Your Pocket), Mar. 2025. [Online]. Available: https://medium.com/data-science-in-your-pocket/google-gemma3-the-best-non-reasoning-llm-e1e017e8da9d (Citato a pagina 39)

Ringraziamenti

A chi è giunto a leggere fin qui, congratulazioni. Dopo il rigore metodologico delle pagine precedenti, le prossime righe seguiranno un ordine dettato solo dal cuore.

Il mio primo ringraziamento va al Professor Fabio Palomba, per aver accettato di essere il relatore di questa tesi. La sua disponibilità e la sua guida sono state preziose e determinati per la riuscita di questa tesi, permettendomi di portare a termine un lavoro a tratti faticoso, altre volte incredibilmente affascinante.

Ringrazio tutti i docenti che ho incontrato in questi anni, poiché ognuno dei loro insegnamenti ha lasciato un segno. Un pensiero speciale va alla Professoressa Annalisa De Bonis: non c'è stato un singolo Design Pattern implementato in questa tesi senza che mi tornassero in mente le sue lezioni di Programmazione Avanzata (o le sue grida).

Un grazie immenso alla mia famiglia, per avermi garantito un clima di serenità indispensabile e per aver permesso di dedicarmi agli studi senza pressioni. Spero di ricompensare sempre il vostro supporto silenzioso.

Un grazie che merita uno spazio tutto suo va a te, mamma. Per ogni comodità che mi hai offerto, per tutte le volte che mi hai detto "dai che ce la fai" quando pensavo di non averne più le forze. Grazie per essere stata in pensiero per me, quando alle nove di sera ero ancora disperso sui mezzi pubblici in qualche parte della Campania. Grazie per aver preferito andare a lavoro a piedi pur di cedermi l'auto, solo per farmi

tornare a casa prima. Grazie per quelle piccole attenzioni, come il letto trovato già fatto, che erano la cura più efficace a ogni giornata storta. Grazie per essere il mio angelo custode.

Un ringraziamento inaspettato, ma non per questo meno sentito, va ai mezzi pubblici. Senza le loro attese infinite, non avrei mai potuto divorare così tanti libri con il mio Kindle, trasformando il tempo perso in tempo guadagnato.

Un ringraziamento ancor più particolare va alla mensa universitaria. Non posso confermare di essere diventato radiattivo ma di certo non ho ancora sviluppato mutazioni visibili. Grazie per avermi *saziato* ogni giorno.

Ringrazio le persone stupende che hanno condiviso con me questo percorso universitario, i momenti di spensieratezza che hanno alleggerito lo studio e anche quelli di angoscia condivisa che ci hanno unito. - 'Carlo Vive'

Un grazie particolare va a te, Raffaele. Per aver condiviso gli infiniti ritorni a casa, per essere stato il testimone di mille peripezie a cui, senza di te, stenterei a credere perfino io. La tua compagnia ha reso sopportabile l'insopportabile.

Infine, permettetemi di ringraziare me stesso. Per la tenacia di persistere, per le sveglie ancor prima dell'alba, per aver sopportato ore estenuanti di ritardi senza scegliere la via più comoda. Per non essermi lasciato abbattere da giornate infinite, trovando sempre la forza di ricominciare il giorno dopo. Ancora oggi mi chiedo come sia riuscito a studiare e superare gli esami in quelle condizioni, ma la risposta, forse, è in qualcosa che oggi mi ha portato fin qui.

Grazie a tutti i presenti.